DOSSIER TECHNIQUE

InfraReady

Projet fil rouge

WIN-DC01
AD DS · DNS · 192.168.50.10

OPS01
Ansible · Prometheus · Grafana · .20

LINUX01
nginx · stockage · .30

Déploiement, automatisation et supervision d'un système d'information Linux/Windows

Administration systèmes Linux/Windows
Bachelor 3 Cybersécurité — Informatique, Infrastructure et Réseau

BINÔME

VICENTE Romain
GRANANA Enzo

DÉPÔT GITHUB

https://github.com/zelttrox/projet-adminsys

Sommaire

1 Besoin et contexte 2

2 Architecture et adressage IP 4

3 Prérequis et installation 7

4 Administration Windows 11

5 Administration Linux 15

6 Automatisation avec Ansible 18

7 Maintenance et mises à jour 21

8 Supervision avec Prometheus et Grafana 25

9 GitHub et intégration continue 29

10 Sécurité et limites connues 32

11 Rapport d'incident 34

12 Matrice critères → preuves 41

59 figures · dépôt https://github.com/zelttrox/projet-adminsys

1 Besoin et contexte

L'entreprise fictive InfraReady exploite des serveurs Linux et Windows configurés en partie à la main. Les procédures ne sont pas standardisées, les mises à jour sont difficiles à suivre et les incidents longs à diagnostiquer.

Mission

La direction informatique demande une plateforme de laboratoire qui permette de **standardiser les configurations**, de rendre les opérations d'administration **reproductibles** et de **surveiller** l'état des serveurs. Le binôme intervient comme équipe d'administration systèmes : concevoir, déployer, automatiser, superviser et documenter une infrastructure Linux/Windows dont les principales opérations sont reproductibles à l'aide de scripts et de playbooks versionnés.

Réponse apportée

Besoin	Réponse dans InfraReady
Configurations standardisées	Chaque configuration (comptes, AD, pare-feu, stockage, supervision) est décrite dans un playbook Ansible versionné
Opérations reproductibles	Playbooks idempotents : un second passage donne `changed=0`. Toute dérive est corrigée en une commande
Mises à jour suivies	Procédure de maintenance automatisée avec contrôles avant/après et rapport daté par machine
Surveillance des serveurs	Prometheus, node_exporter, windows_exporter, tableau de bord Grafana et 8 règles d'alerte
Diagnostic des incidents	Métriques, alertes et journaux (sudo, ufw, sécurité Windows) exploités selon une méthode par couches
Aucun secret exposé	Ansible Vault, contrôle automatique des secrets, gitleaks sur tout l'historique : aucune fuite

Synthèse des résultats

Partie du sujet	Statut	Éléments principaux
A. Installation et réseau	Atteint	3 VM, réseau NAT dédié, adresses fixes, DNS du domaine testé dans les deux sens
B. Administration Windows	Atteint	Domaine `infraready.test`, DNS, 2 groupes et 6 comptes, journaux de sécurité vérifiés
C. Administration Linux	Atteint	nginx, comptes et permissions, délégation sudo testée, disque `/srv/data` persistant
D. Automatisation Ansible	Atteint	10 playbooks, connexions Linux (SSH) et Windows (WinRM HTTPS), Vault, idempotence prouvée
E. Maintenance	Atteint	Procédure documentée et automatisée, appliquée sur les 3 machines
F. Supervision	Atteint	4 cibles, 12 métriques, tableau de bord, alerte démontrée

Partie du sujet	Statut	Éléments principaux
G. GitHub et CI	Partiel	Dépôt et workflow prêts ; exécution sur GitHub bloquée par le compte, contrôles rejoués en local
H. Diagnostic d'incident	Atteint	Incident injecté et diagnostiqué, plus trois incidents réels documentés

RÉFÉRENCE

Dépôt GitHub du binôme : **https://github.com/zelttrox/projet-adminsys**. Chaque élément de ce dossier y est versionné (playbooks, configurations de supervision, documentation `docs/`, rapports `preuves/`).

2 Architecture et adressage IP

Trois machines virtuelles sur VMware Workstation Pro, dans un réseau NAT dédié 192.168.50.0/24, autour du domaine infraready.test.

Schéma d'architecture

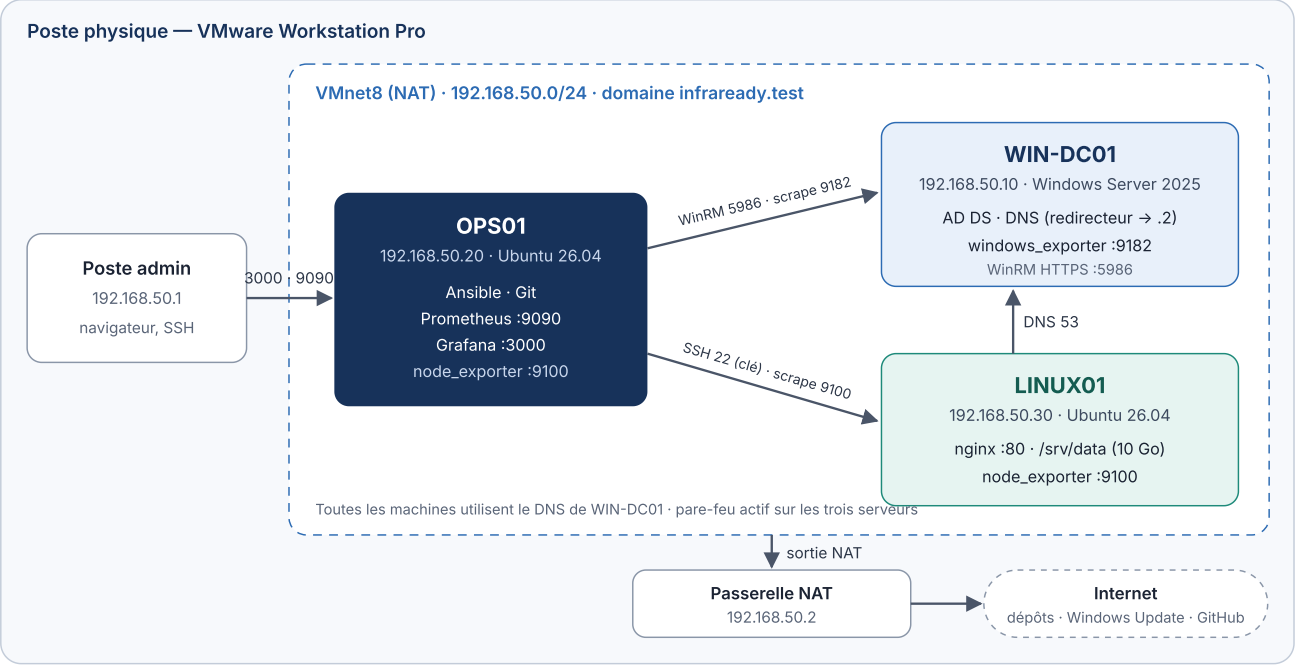


Schéma 1. Architecture du laboratoire et flux principaux. Les flèches indiquent le sens d'ouverture des connexions.

Rôles des machines

Machine	Système	Rôle	Services
WIN-DC01	Windows Server 2025	Annuaire et noms	AD DS (forêt `infraready.test`), DNS (zones directe et inverse, redirecteur), windows_exporter
OPS01	Ubuntu Server 26.04	Administration et supervision	Ansible, Git (dépôt `~/infraready`), Prometheus, Grafana, node_exporter
LINUX01	Ubuntu Server 26.04	Services Linux	nginx, disque de données `/srv/data`, comptes et groupes, node_exporter

Plan d'adressage

Paramètre	Valeur
Réseau virtuel	VMnet8 (NAT), sous-réseau `192.168.50.0/24`
Poste physique	`192.168.50.1` (interface `vmnet8`)
Passerelle	`192.168.50.2` (NAT VMware, sortie Internet pour les mises à jour)
DNS du laboratoire	`192.168.50.10` (WIN-DC01), redirecteur vers `192.168.50.2`
Domaine	`infraready.test` (NetBIOS `INFRAREADY`)

Machine	Adresse	Nom DNS (A + PTR)	Configuration
WIN-DC01	`192.168.50.10/24`	`win-dc01.infraready.test`	IPv4 statique, DNS `127.0.0.1` après promotion
OPS01	`192.168.50.20/24`	`ops01.infraready.test`	Netplan statique, DNS `192.168.50.10`
LINUX01	`192.168.50.30/24`	`linux01.infraready.test`	Netplan statique, DNS `192.168.50.10`

Matrice des flux autorisés

Tout flux entrant non listé est refusé par les pare-feux des trois machines (section 10).

Destination	Port	Source	Usage
LINUX01	22/tcp	192.168.50.0/24	SSH (Ansible, poste admin)
LINUX01	80/tcp	192.168.50.0/24	Site nginx
LINUX01	9100/tcp	**192.168.50.20 seulement**	node_exporter → Prometheus
OPS01	22, 3000, 9090/tcp	192.168.50.0/24	SSH, Grafana, Prometheus
WIN-DC01	5986/tcp	**192.168.50.20 seulement**	WinRM HTTPS (Ansible)
WIN-DC01	9182/tcp	**192.168.50.20 seulement**	windows_exporter → Prometheus
WIN-DC01	53, 88, 389…	Réseau du labo	DNS et Active Directory (règles des rôles)

Choix techniques

Choix	Justification
Trois VM séparées	Les ressources du poste le permettent. La panne de LINUX01 n'affecte ni l'administration ni la supervision
Réseau NAT dédié	Mises à jour possibles par la passerelle VMware, sans exposer de service sur le réseau de l'établissement
Adresses fixes	Un contrôleur de domaine et des cibles de supervision ne doivent pas changer d'adresse
Inventaire Ansible par noms DNS	Le même inventaire fonctionne dans les labos des deux membres du binôme
Prometheus par adresses IP	Le DNS est porté par WIN-DC01 : si le contrôleur tombe, LINUX01 et OPS01 restent supervisés
WinRM HTTPS et SSH par clé	Transport chiffré vers les deux familles de systèmes, sans agent à installer

Écarts par rapport au sujet

Écart	Conséquence	Traitement
Ubuntu 26.04 au lieu de 24.04	sudo-rs par défaut, Python 3.14, collections absentes d'`ansible-core`	`ansible_become_exe=sudo.ws`, `collections/requirements.yml`

Écart	Conséquence	Traitement
LINUX01 en .30 et OPS01 en .20 (inversés)	Aucune	Documenté dans `docs/adressage-ip.md`
Inventaire `hosts.ini` au lieu de `hosts.yml`	Aucune	Format INI natif d'Ansible
GitHub Actions non exécuté	Pas de validation côté GitHub	Contrôles rejoués en local (section 9)

3 Prérequis et installation

Mise en place du réseau virtuel, des trois serveurs, du DNS du domaine et des accès d'administration nécessaires à Ansible.

Environnement

Élément	Valeur
Hyperviseur	VMware Workstation Pro 26H1
WIN-DC01	Windows Server 2025 Standard (évaluation), disque C: 48 Go
OPS01	Ubuntu Server 26.04, 5 Go de RAM, 4 Go de swap
LINUX01	Ubuntu Server 26.04, 3,3 Go de RAM, disque système 30 Go + disque de données 10 Go
Snapshots	Avant chaque changement important (promotion AD, mises à jour, pare-feu, stockage)

Réseau virtuel et adresses

VMnet8 est configuré en NAT sur `192.168.50.0/24` avec la passerelle en `.2`. L'absence de conflit avec les réseaux du poste a été vérifiée avant usage. Chaque serveur reçoit une adresse fixe.

```
~ ❯ ip addr show vmnet8
14: vmnet8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group defa
ult qlen 1000
    link/ether 00:50:56:c0:00:08 brd ff:ff:ff:ff:ff:ff
    altname enx005056c00008
    inet 192.168.50.1/24 brd 192.168.50.255 scope global vmnet8
       valid_lft forever preferred_lft forever
    inet6 fe80::250:56ff:fec0:8/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
```

Figure 1. Interface `vmnet8` du poste en `192.168.50.1/24`.

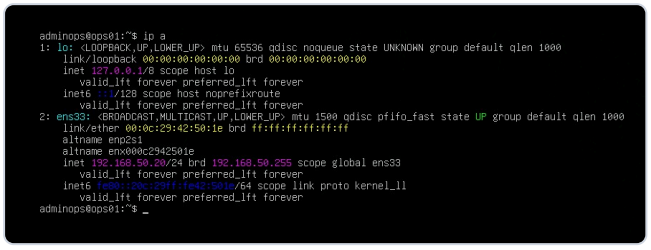


Figure 2. OPS01 en `192.168.50.20/24` sur `ens33`.

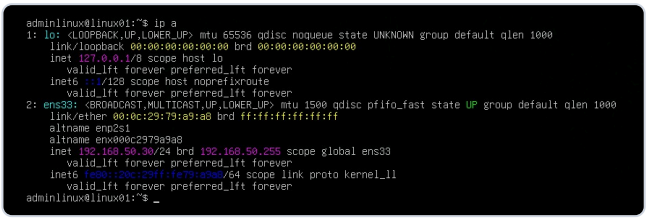


Figure 3. LINUX01 en `192.168.50.30/24` sur `ens33`.

```
Suffixe DNS propre à la connexion. . . :
Description. . . . . . . . . . . . . . : Intel(R) 82574L Gigabit Network Connection
Adresse physique . . . . . . . . . . . : 00-0C-29-19-2C-E6
DHCP activé. . . . . . . . . . . . . . : Non
Configuration automatique activée. . . : Oui
Adresse IPv6 de liaison locale. . . . .: fe80::5195:f7b1:e072:e8ae%12(préféré)
Adresse IPv4. . . . . . . . . . . . . .: 192.168.50.10(préféré)
Masque de sous-réseau. . . . . . . . . : 255.255.255.0
Passerelle par défaut. . . . . . . . . : 192.168.50.2
```

Figure 4. WIN-DC01 en adresse fixe `192.168.50.10`, DHCP désactivé, passerelle `192.168.50.2`.

Promotion du contrôleur de domaine

Le rôle AD DS est installé puis la forêt est créée avec le DNS intégré :

PowerShell (administrateur) sur WIN-DC01

```
Install-WindowsFeature AD-Domain-Services -IncludeManagementTools
Install-ADDSForest -DomainName "infraready.test" -DomainNetbiosName "INFRAREADY" `
                   -InstallDns -SafeModeAdministratorPassword (...) -Force
```

Configuration DNS complémentaire : redirecteur vers `192.168.50.2` pour la résolution Internet, zone inversée `50.168.192.in-addr.arpa`, enregistrements A et PTR d'OPS01 et de LINUX01.

DIFFICULTÉS RENCONTRÉES

Deux blocages rencontrés et résolus : (1) la promotion refusait de démarrer car le compte Administrateur local n'avait pas de mot de passe — il devient l'administrateur du domaine et doit en avoir un ; (2) la gestion DNS renvoyait « accès refusé » depuis une console PowerShell non élevée (contrôle de compte d'utilisateur).

DNS des serveurs Linux

Les deux Ubuntu utilisaient initialement la passerelle `192.168.50.2` comme DNS et ne résolvaient donc pas le domaine. Le fichier Netplan a été corrigé pour pointer sur WIN-DC01, et la gestion du réseau par cloud-init désactivée pour que la correction survive aux redémarrages.

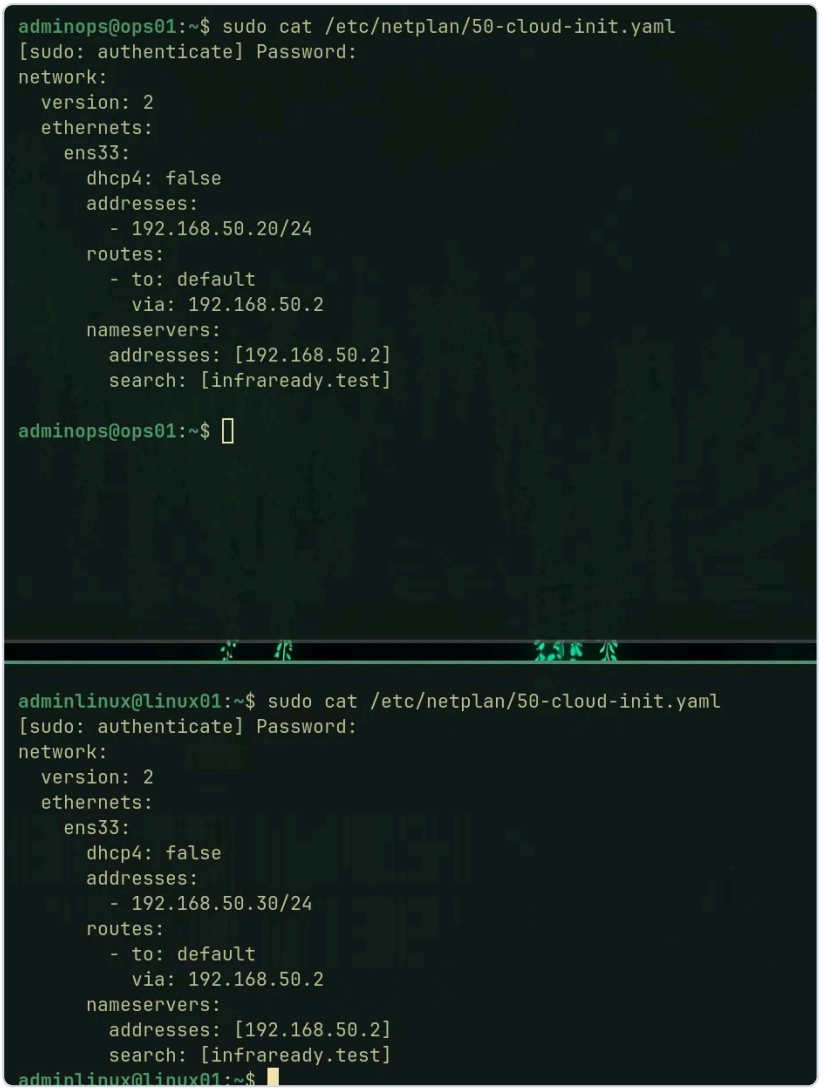


Figure 5. Configuration Netplan initiale d'OPS01 et de LINUX01 : DNS sur la passerelle `192.168.50.2`.

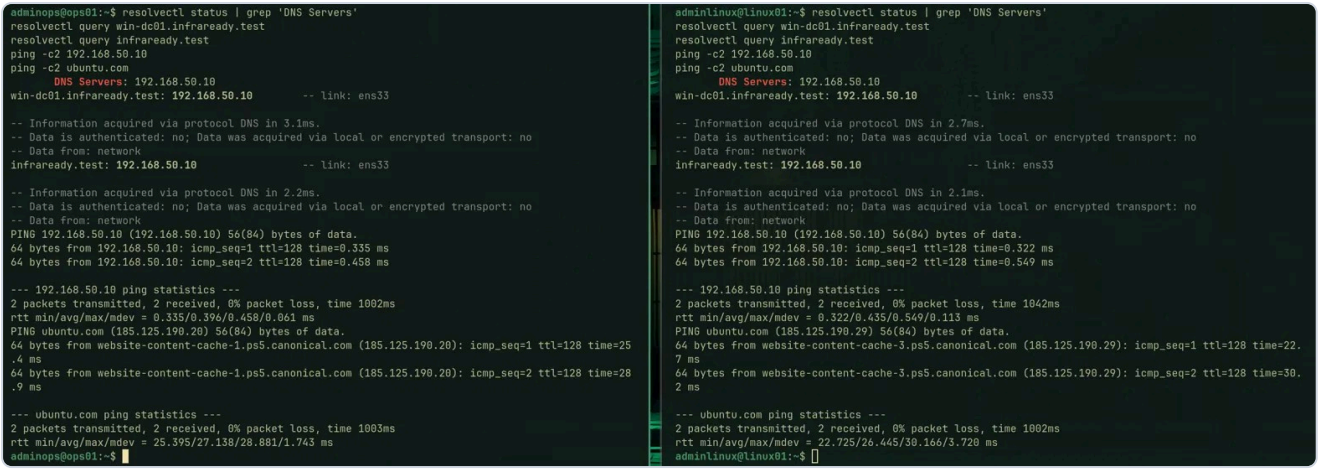


Figure 6. Après correction : DNS `192.168.50.10` sur les deux machines, `win-dc01.infraready.test` et `infraready.test` résolus, ping du contrôleur et d'Internet réussis.

Accès d'administration

OPS01 se connecte à LINUX01 par **clé SSH ed25519**, sans mot de passe, et à WIN-DC01 par **WinRM HTTPS** (port 5986) avec une règle de pare-feu limitée à OPS01. Ansible et les collections Windows, AD et Linux sont installés sur OPS01.

```
adminops@ops01:~$ ssh adminlinux@192.168.50.30 'hostname; whoami; echo OK sans mot de passe'
linux01
adminlinux
OK sans mot de passe
adminops@ops01:~$ ssh adminlinux@linux01.infraready.test 'hostname'
The authenticity of host 'linux01.infraready.test (192.168.50.30)' can't be established.
ED25519 key fingerprint is: SHA256:AJh3ayr0fX8mqWaaUFc67TGaKNCwEmu3Vx5+8PiOrqM
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'linux01.infraready.test' (ED25519) to the list of known hosts.
linux01
```

Figure 7. Connexion SSH par clé d'OPS01 vers LINUX01, par adresse puis par nom DNS.

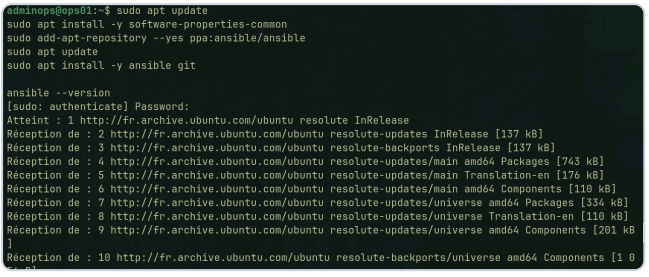


Figure 8. Installation d'Ansible sur OPS01 (dépôt « resolute », Ubuntu 26.04).

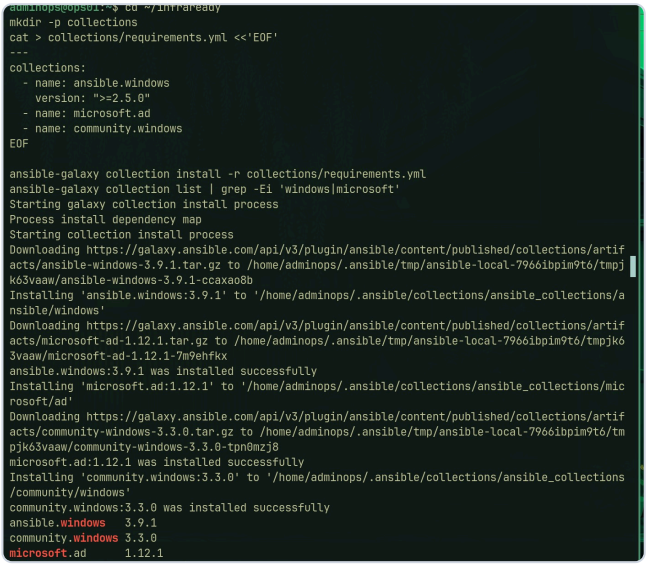


Figure 9. Installation des collections `ansible.windows`, `microsoft.ad`, `community.windows`.

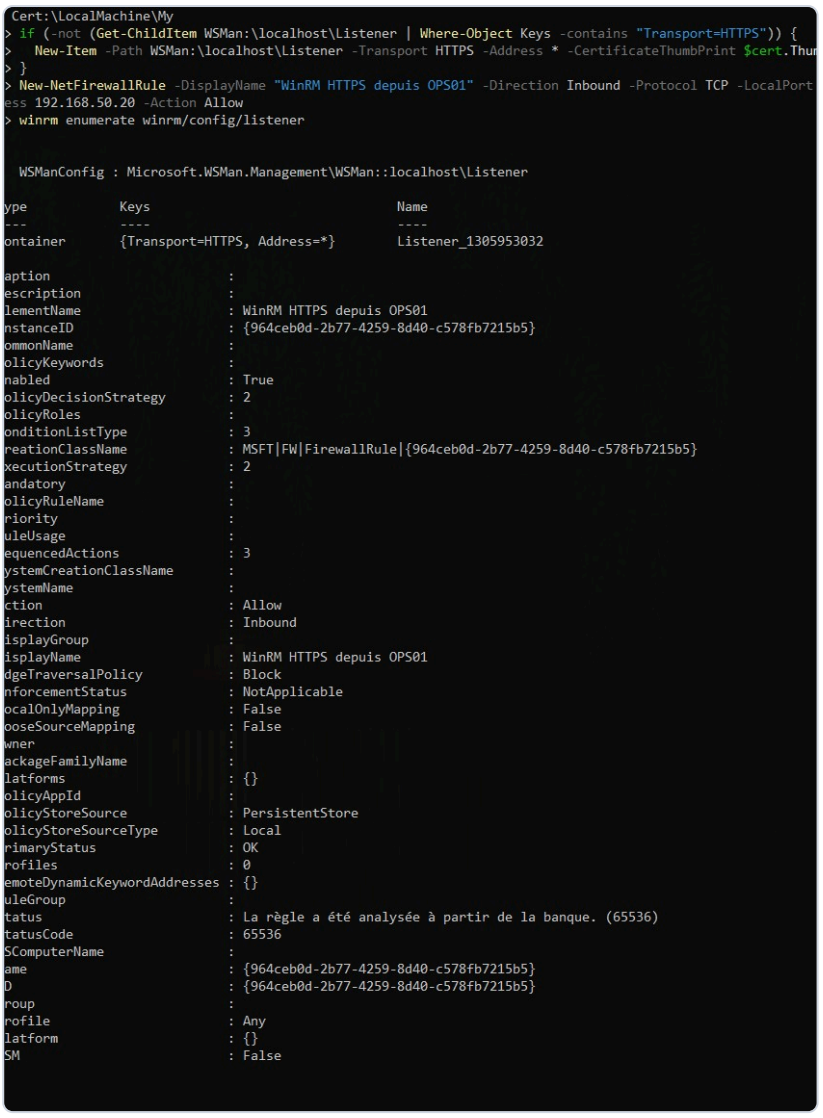


Figure 10. Activation du listener WinRM HTTPS et de la règle de pare-feu « WinRM HTTPS depuis OPS01 ».

4 Administration Windows

Domaine Active Directory opérationnel, DNS testé, structure d'OU, groupes et comptes créés par automatisation, journaux vérifiés.

Domaine et DNS

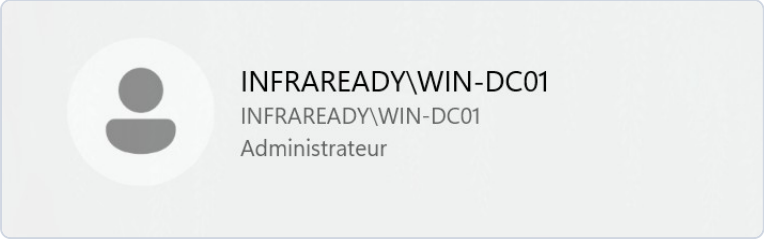


Figure 11. Session ouverte sur le domaine `INFRAREADY`.



Figure 12. Compte d'administration `WIN-DC01@infraready.test`.

```
PS C:\WINDOWS\system32> Get-DnsClientServerAddress -AddressFamily IPv4

InterfaceAlias               Interface Address ServerAddresses
                             Index     Family
--------------               --------- ------- ---------------
Ethernet0                            4 IPv4    {127.0.0.1}
Loopback Pseudo-Interface 1          1 IPv4    {}
```

Figure 13. Le contrôleur utilise son propre service DNS (`127.0.0.1`).

```
>> Get-DnsServerResourceRecord -ZoneName 50.168.192.in-addr.arpa | Format-Table HostName, RecordData

HostName RecordData
-------- ----------
LINUX01  DnsServerResourceRecordA
OPS01    DnsServerResourceRecordA



HostName RecordData
-------- ----------
@        DnsServerResourceRecordNS
@        DnsServerResourceRecordSoa
20       DnsServerResourceRecordPtr
30       DnsServerResourceRecordPtr
```

Figure 14. Enregistrements A de LINUX01 et OPS01 dans `infraready.test` et pointeurs PTR `20` et `30` dans la zone inversée.

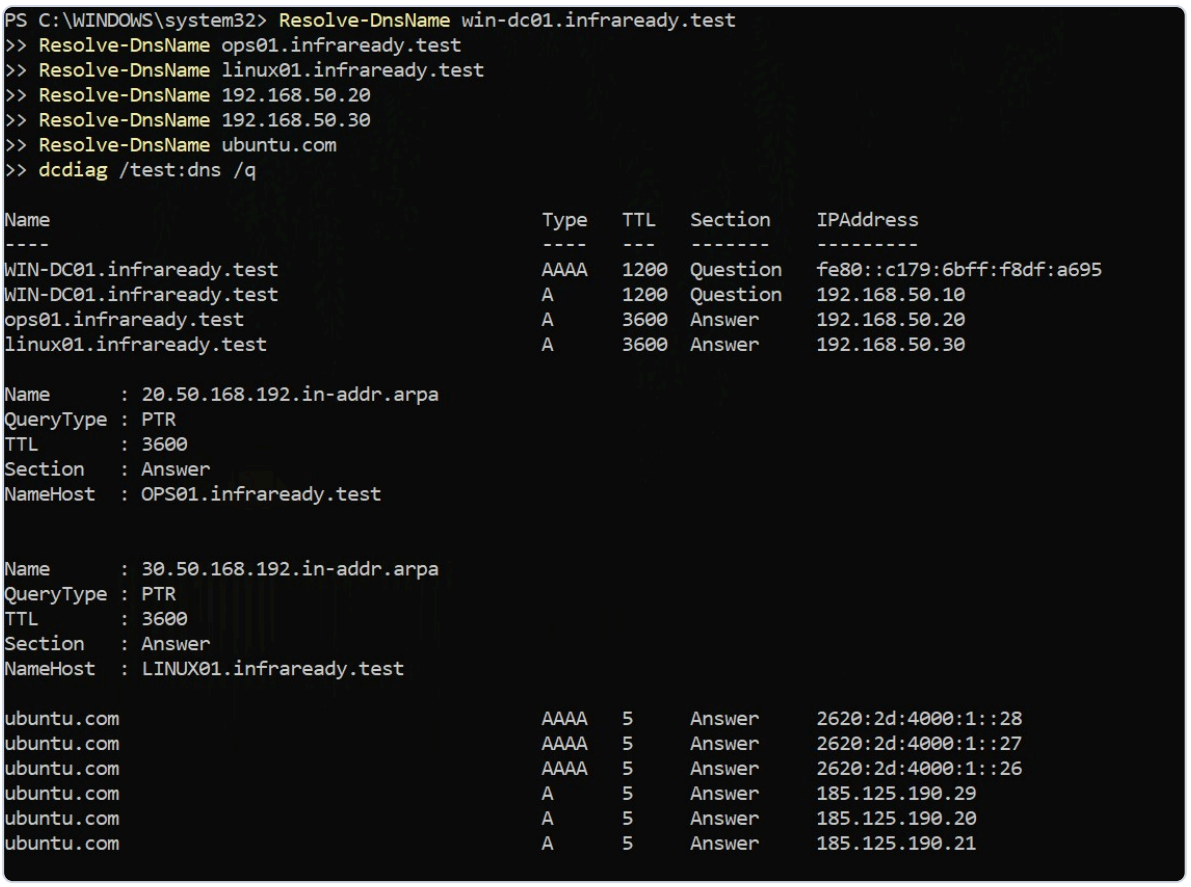


Figure 15. Tests de réception : résolution directe des trois noms, résolution inverse de `.20` et `.30`, résolution Internet via le redirecteur, et `dcdiag /test:dns /q` sans aucune erreur.

Organisation, groupes et comptes

La structure est créée par le script `scripts/windows/New-InfraReadyADObjects.ps1`, déployé et exécuté par le playbook `playbooks/windows/ad-groupes-utilisateurs.yml`. Le script est **idempotent** : chaque OU, groupe, compte et appartenance est vérifié avant création (par nom, SamAccountName et UPN pour les comptes). Le mot de passe initial provient d'Ansible Vault et doit être changé à la première connexion.

Objet	Emplacement	Contenu
OU `InfraReady`	`infraready.test`	Racine des objets du projet
OU `Utilisateurs`	`InfraReady`	6 comptes
OU `Groupes`	`InfraReady`	2 groupes de sécurité globaux
Groupe **Employés**	`InfraReady/Groupes`	`amartin` (Comptabilité), `bpetit` (RH), `cdurand` (Commercial)
Groupe **IT**	`InfraReady/Groupes`	`dleroy`, `emoreau`, `fbenali` (Informatique)

```
TASK [Executer le script de creation des objets AD] ****************************************
changed: [win-dc01]

TASK [Afficher le compte-rendu du script] **************************************************
ok: [win-dc01] => {
    "ad_creation.output": [
        "[CREE    ] OU           InfraReady",
        "[CREE    ] OU           Utilisateurs",
        "[CREE    ] OU           Groupes",
        "[CREE    ] Groupe       Employes",
        "[CREE    ] Groupe       IT",
        "[CREE    ] Utilisateur  amartin",
        "[AJOUTE  ] Membre       amartin -> Employes",
        "[CREE    ] Utilisateur  bpetit",
        "[AJOUTE  ] Membre       bpetit -> Employes",
        "[CREE    ] Utilisateur  cdurand",
        "[AJOUTE  ] Membre       cdurand -> Employes",
        "[CREE    ] Utilisateur  dleroy",
        "[AJOUTE  ] Membre       dleroy -> IT",
        "[CREE    ] Utilisateur  emoreau",
        "[AJOUTE  ] Membre       emoreau -> IT",
        "[CREE    ] Utilisateur  fbenali",
        "[AJOUTE  ] Membre       fbenali -> IT",
        "Bilan : 11 cree(s), 0 deja existant(s), 0 erreur(s)"
    ]
}

TASK [Lire les groupes et leurs membres] ***************************************************
ok: [win-dc01]

TASK [Controler la presence des groupes et le nombre de membres] ***************************
ok: [win-dc01] => {
    "changed": false,
    "msg": "Les groupes Employes, IT ont chacun au moins 3 membres"
}

TASK [Afficher l'etat final] ***************************************************************
ok: [win-dc01] => (item=Employes) => {
    "msg": "Employes (3) : amartin, bpetit, cdurand"
}
ok: [win-dc01] => (item=IT) => {
    "msg": "IT (3) : dleroy, emoreau, fbenali"
}

PLAY RECAP *********************************************************************************
win-dc01                   : ok=9    changed=3    unreachable=0    failed=0    skipped=0    rescued=0
  ignored=0
```

Figure 16. Exécution du playbook : 11 objets créés, appartenances ajoutées, contrôle automatique « au moins 3 membres par groupe ».

```
PS C:\WINDOWS\system32> Get-ADGroupMember Employes | Select Name, SamAccountName
>> Get-ADGroupMember IT | Select Name, SamAccountName

Name         SamAccountName
----         --------------
Alice Martin amartin
Bruno Petit  bpetit
Chloe Durand cdurand
David Leroy  dleroy
Emma Moreau  emoreau
Farid Benali fbenali


PS C:\WINDOWS\system32>
```

Figure 17. Vérification sur le contrôleur : les 6 comptes membres des groupes Employés et IT.

Droits et accès accordés

Compte ou groupe	Droits	Justification
Comptes Employés et IT	Utilisateurs du domaine, sans privilège. Mot de passe à changer à la première connexion	Moindre privilège : les droits seront délégués aux groupes, jamais aux comptes

Compte ou groupe	Droits	Justification
Groupes Employés et IT	Groupes de sécurité globaux, prêts à recevoir des permissions (partages, GPO)	Gestion des accès par groupe, et non compte par compte
`WIN-DC01@infraready.test`	Administrateur du domaine, utilisé par Ansible via WinRM HTTPS	Nécessaire pour créer OU, groupes et comptes. Limite documentée en section 10
WinRM 5986 / exporter 9182	Accessibles uniquement depuis OPS01	Surface d'administration réduite à la machine d'administration

Journaux Windows

Les créations sont tracées dans le journal Sécurité : **4727** (création de groupe de sécurité global), **4720** (création de compte), **4728** (ajout à un groupe). Les 14 événements tombent en deux secondes, signature d'une exécution automatisée. Les journaux *Directory Service* et *DNS Server* ne contiennent que des événements de niveau Information.

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4720,4728,4727} -MaxEvents 15 |
>>   Select TimeCreated, Id, @{n='Message';e={($_.Message -split "`n")[0]}} | Format-Table -AutoSize
>> Get-WinEvent -LogName 'Directory Service' -MaxEvents 5 | Select TimeCreated, LevelDisplayName, Id
>> Get-WinEvent -LogName 'DNS Server' -MaxEvents 5 | Select TimeCreated, LevelDisplayName, Id

TimeCreated           Id Message
-----------           -- -------
22/10/2026 15:51:08 4728 Un membre a été ajouté à un groupe global dont la sécurité est activé...
22/10/2026 15:51:08 4720 Un compte d'utilisateur a été créé....
22/10/2026 15:51:07 4728 Un membre a été ajouté à un groupe global dont la sécurité est activé...
22/10/2026 15:51:07 4720 Un compte d'utilisateur a été créé....
22/10/2026 15:51:07 4728 Un membre a été ajouté à un groupe global dont la sécurité est activé...
22/10/2026 15:51:07 4720 Un compte d'utilisateur a été créé....
22/10/2026 15:51:07 4728 Un membre a été ajouté à un groupe global dont la sécurité est activé...
22/10/2026 15:51:07 4720 Un compte d'utilisateur a été créé....
22/10/2026 15:51:07 4728 Un membre a été ajouté à un groupe global dont la sécurité est activé...
22/10/2026 15:51:07 4720 Un compte d'utilisateur a été créé....
22/10/2026 15:51:07 4728 Un membre a été ajouté à un groupe global dont la sécurité est activé...
22/10/2026 15:51:06 4720 Un compte d'utilisateur a été créé....
22/10/2026 15:51:06 4727 Un groupe global dont la sécurité est activée a été créé....
22/10/2026 15:51:06 4727 Un groupe global dont la sécurité est activée a été créé....
21/10/2026 16:28:05 4727 Un groupe global dont la sécurité est activée a été créé....



TimeCreated         LevelDisplayName   Id
-----------         ---------------   --
22/10/2026 15:22:50 Information      1869
22/10/2026 15:22:50 Information      3027
22/10/2026 15:22:50 Information      3033
22/10/2026 15:08:20 Information      1394
22/10/2026 15:07:50 Information      1000
22/10/2026 15:08:21 Information         4
22/10/2026 15:08:21 Information         2
22/10/2026 15:08:20 Information       769
22/10/2026 15:08:20 Information       769
22/10/2026 15:08:20 Information       769
```

Figure 18. Journaux Sécurité, Directory Service et DNS Server après la création des objets.

```
                    Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4727; EndTime='2026-10-21 23:59'} -MaxEvent
s 1 |
>>   ForEach-Object { ([xml]$_.ToXml()).Event.EventData.Data | Where-Object Name -in 'TargetUserName','SubjectUserName' }

Name           #text
----           -----
TargetUserName  DnsUpdateProxy
SubjectUserName WIN-DC01$
```

Figure 19. L'événement 4727 du 21/10 concerne le groupe système `DnsUpdateProxy`, créé par le compte machine `WIN-DC01$` lors de l'installation du rôle DNS : action système et non d'administration.

5 Administration Linux

Service web, comptes et permissions selon le besoin, délégation sudo minimale, disque de données persistant, services, processus et journaux.

Service applicatif : nginx

nginx est installé par `playbooks/linux/nginx.yml` puis configuré par `comptes-permissions.yml` : site InfraReady servi depuis `/var/www/infraready`, site par défaut désactivé, rechargement uniquement si la configuration change.



Figure 20. nginx actif et activé au démarrage, processus maître (root) et workers, aucun service en échec.

Comptes, groupes et permissions

Élément	Configuration	Justification
Groupe `webadmins`	`webdev1`, `webdev2`	Publient le contenu du site
Groupe `support`	`support1` (également dans `adm`)	Consulte le site et les journaux

Élément	Configuration	Justification
`/var/www/infraready`	`root:webadmins`, mode **2775**	Le bit setgid fait hériter le groupe : tout webadmin peut modifier le travail d'un autre
Comptes	Mot de passe verrouillé, `/home` en 750	Connexion par clé uniquement, pas de lecture croisée
sudo du groupe `support`	`systemctl status nginx` et `systemctl restart nginx` uniquement	Délégation minimale, fichier validé par `visudo` avant écriture

Le playbook vérifie lui-même quatre comportements à chaque exécution : `webdev1` peut écrire, le fichier créé hérite du groupe `webadmins`, `support1` ne peut **pas** écrire, `support1` peut lire les journaux nginx.



Figure 21. Premier passage : `changed=10`.



Figure 22. Second passage : `changed=0` (idempotence).

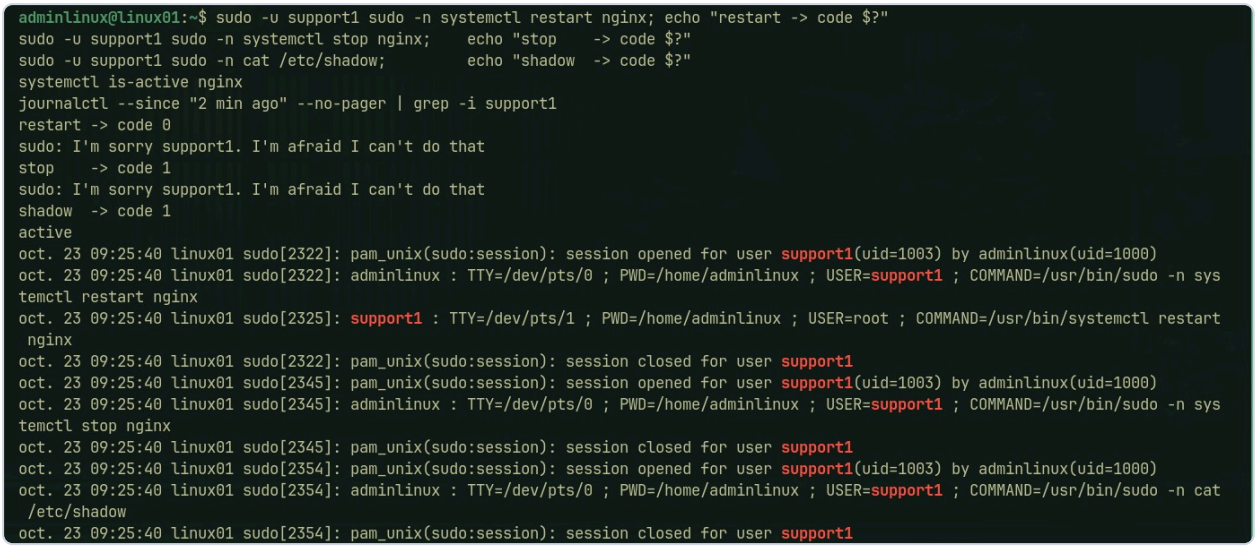


Figure 23. Test de la délégation : `restart` autorisé (code 0), `stop` et `cat /etc/shadow` refusés (code 1), nginx toujours actif. Le journal attribue chaque commande au bon compte.

Stockage

Un disque virtuel de 10 Go est ajouté à LINUX01 puis préparé par `playbooks/linux/stockage.yml` : table GPT, partition unique, système de fichiers ext4 étiqueté `DATA`, montage sur `/srv/data` **par UUID** avec l'option `nofail`, droits `root:webadmins` 2770. Le playbook refuse de travailler si le disque n'est pas fourni explicitement, s'il s'agit du disque système ou s'il contient déjà des données.

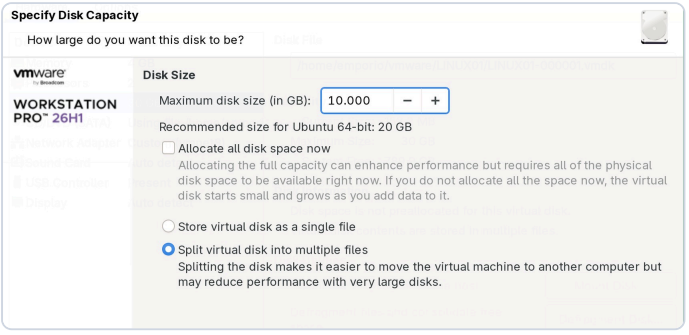


Figure 24. Ajout du disque virtuel de 10 Go.

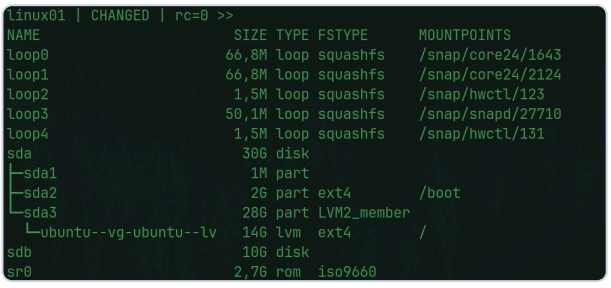


Figure 25. Le disque apparaît en `sdb`, vide ; `sda` porte le système (LVM).

```
adminops@ops01:~/infraready$ ansible linux -m reboot -K --become
ansible linux -m command -a "findmnt /srv/data"
ansible linux -m command -a "grep /srv/data /etc/fstab"
BECOME password:
linux01 | CHANGED => {
    "changed": true,
    "elapsed": 37,
    "rebooted": true
}
linux01 | CHANGED | rc=0 >>
TARGET    SOURCE    FSTYPE OPTIONS
/srv/data /dev/sdb1 ext4   rw,relatime
linux01 | CHANGED | rc=0 >>
UUID=51f87387-5a24-48c8-8b66-4660a65eb5ca /srv/data ext4 defaults,nofail 0 0
```

Figure 26. Preuve de persistance : après un redémarrage, `/srv/data` est remonté automatiquement depuis `/dev/sdb1` (ext4) grâce à la ligne fstab par UUID.

Services, processus et journaux

Action	Commandes	Résultat
État des services	`systemctl status|is-enabled|is-active`, `systemctl --failed`	nginx `enabled` et `active`, aucun service en échec
Processus et ports	`ps -C nginx`, `ss -tlnp`	Maître nginx en root (port 80), workers en `www-data`
Processus gourmand	`top`, `pgrep`, `renice +19`, `kill`	Priorité abaissée puis arrêt propre (SIGTERM)
Journaux	`journalctl -u nginx`, `-p err -b`, `-t sudo`, `--list-boots`	Journal persistant entre les redémarrages, traçabilité des commandes sudo

La mise à jour contrôlée du système est traitée avec la maintenance (section 7).

6 Automatisation avec Ansible

OPS01 pilote les trois machines : Linux en SSH par clé, Windows en WinRM HTTPS. Les secrets sont dans Ansible Vault.

Configuration

```
adminops@ops01:~/infraready$ cat ansible.cfg
[defaults]
inventory = inventory/hosts.ini
roles_path = roles
host_key_checking = False
interpreter_python = auto_silent
nocows = 1
```

Figure 27. Fichier `ansible.cfg` à la racine du dépôt.

inventory/hosts.ini

```
[windows]
win-dc01 ansible_host=win-dc01.infraready.test
[linux]
linux01 ansible_host=linux01.infraready.test
[ops]
ops01 ansible_connection=local

[windows:vars]
ansible_user=WIN-DC01@infraready.test
ansible_connection=winrm
ansible_port=5986
ansible_winrm_transport=ntlm
ansible_winrm_server_cert_validation=ignore

[linux:vars]
ansible_user=adminlinux
ansible_become_exe=sudo.ws          # Ubuntu 26.04 : sudo-rs incompatible avec Ansible
ansible_python_interpreter=/usr/bin/python3
```

Secrets : Ansible Vault

Fichier (chiffré AES-256)	Variables	Utilisées par
`group_vars/windows/vault.yml`	`vault_windows_password`, `vault_ad_initial_password`	Connexion WinRM, création des comptes AD
`group_vars/linux/vault.yml`	`vault_become_linux`	Élévation sudo sur LINUX01
`group_vars/ops/vault.yml`	`vault_become_ops`, `vault_grafana_admin`	Élévation sudo sur OPS01, compte admin Grafana

Les fichiers `vars.yml` voisins ne contiennent que des références, par exemple `ansible_password: "{{ vault_windows_password }}"`. Un seul mot de passe de vault, fort et distinct de tous ceux qu'il protège, déverrouille les trois fichiers (`--ask-vault-pass`).

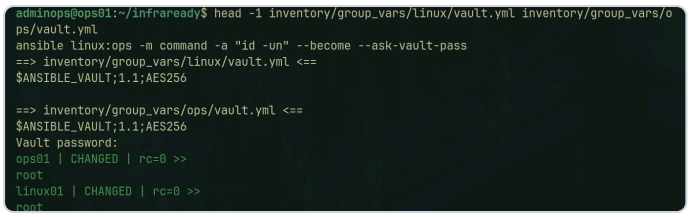


Figure 28. Vaults chiffrés ; Ansible obtient root sur les deux Linux grâce aux mots de passe sudo du vault.

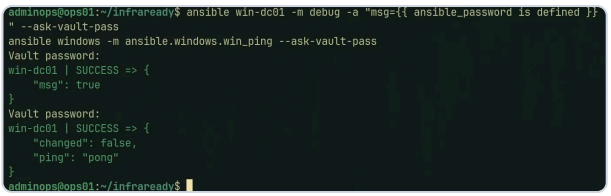


Figure 29. Le mot de passe Windows est bien défini depuis le vault ; `win_ping` répond.

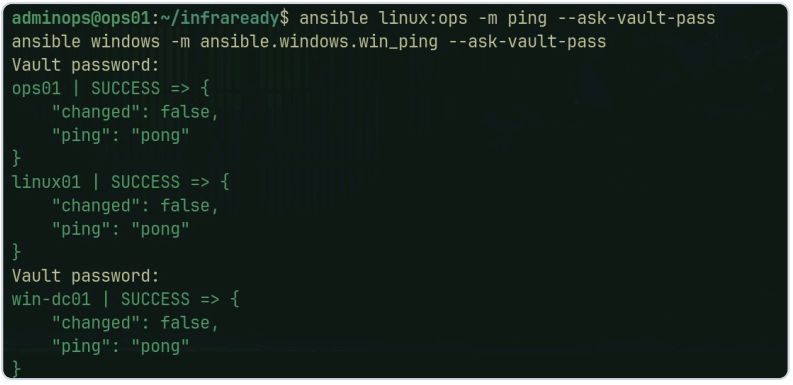


Figure 30. Les trois machines répondent avec un seul mot de passe de vault.

Playbooks

Playbook	Cible	Rôle	Idempotence
`ping-all.yml`	Toutes	Joignabilité des 3 machines	—
`linux/nginx.yml`	LINUX01	Installation et démarrage de nginx	`changed=0` au 2e passage
`linux/comptes-permissions.yml`	LINUX01	Groupes, comptes, setgid, site nginx, sudo délégué + 4 tests	10 → 0
`linux/stockage.yml`	LINUX01	Partition, ext4, montage par UUID, garde-fous	`changed=0` au 2e passage
`windows/ad-groupes-utilisateurs.yml`	WIN-DC01	OU, groupes, comptes AD + contrôle	`changed=0` au 2e passage
`linux/maintenance.yml`	LINUX01, OPS01	Contrôles, mises à jour, rapport	Rapport réécrit à chaque passage
`windows/maintenance-windows.yml`	WIN-DC01	Windows Update natif, contrôles AD/DNS	Rapport réécrit à chaque passage
`monitoring.yml`	Toutes	Exporters, Prometheus, Grafana + vérification	`changed=0` au 2e passage
`pare-feu.yml`	Toutes	ufw et pare-feu Windows + tests de refus	`changed=0` au 2e passage

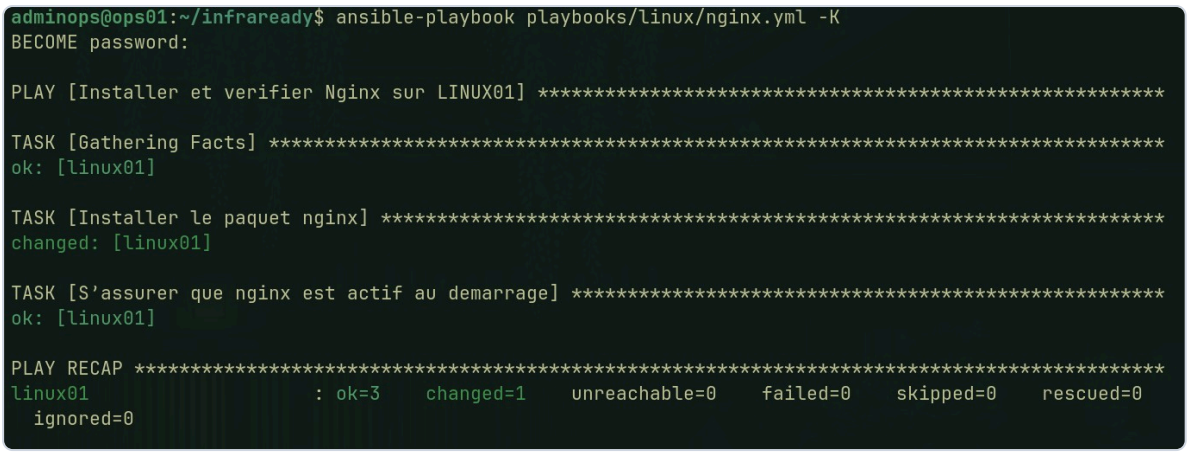


Figure 31. Premier playbook exécuté sur LINUX01 : nginx installé (`changed=1`).

DIFFICULTÉ RENCONTRÉE

L'élévation de privilèges échouait par délai dépassé : Ubuntu 26.04 utilise **sudo-rs**, dont l'invite de mot de passe n'est pas reconnue par Ansible. Solution : `ansible_become_exe=sudo.ws`, qui utilise le sudo classique toujours présent (annexe A3).

7 Maintenance et mises à jour

Procédure documentée dans docs/maintenance.md et automatisée par deux playbooks, avec contrôles avant/après et rapport daté par machine.

Principes de la procédure

Principe	Mise en œuvre
Voir avant d'agir	Mode `rapport` par défaut : liste des mises à jour, aucune installation
Pouvoir revenir en arrière	Le mode `appliquer` est refusé sans `-e snapshot_ok=oui` (snapshot VMware pris)
Ne pas aggraver un système dégradé	Arrêt avant toute modification si un disque dépasse 85 % ou si un service critique est arrêté
Une machine à la fois	`serial: 1` : un échec n'atteint pas les machines suivantes ; LINUX01 avant OPS01, WIN-DC01 en dernier
Redémarrer seulement si nécessaire	Uniquement si le système le demande **et** si `autoriser_reboot=true` ; jamais le contrôleur Ansible lui-même
Protéger l'outillage	Simulation `apt-get -s` et refus si un paquet protégé (`ansible-core`, `nginx`, `prometheus`…) serait supprimé
Prouver le résultat	Contrôles après intervention et rapport `preuves/E_maintenance_<hôte>_<date>.txt`

Linux : LINUX01 et OPS01

```
PLAY RECAP *********************************************************************************
linux01                    : ok=20   changed=4    unreachable=0    failed=0    skipped=2    resc
ued=0    ignored=0
ops01                      : ok=20   changed=3    unreachable=0    failed=0    skipped=2    resc
ued=0    ignored=0
```

Figure 32. Mode rapport sur les deux machines : contrôles réussis, installation et redémarrage sautés.

```
Espace disque (seuil 85 %)
  / : 47.1 % utilise, 7.2 Go libres
  /boot : 11.2 % utilise, 1.7 Go libres
  /srv/data : 5.3 % utilise, 9.2 Go libres

Services critiques apres intervention
  ssh : running
  nginx : running
```

Figure 33. Extrait du rapport de LINUX01 : disques sous le seuil, services actifs.

```
Mises a jour disponibles avant intervention : 48
  - ca-certificates
  - cloud-init-base
  - cloud-init
  - distro-info-data
  - dmidecode
  - libaudit-common
  - libaudit1
  - libflashrom1
  - liblzma5
  - libnetplan1
  - libnss-systemd
  - libnss3
  - libopeniscsiusr
  - libpam-systemd
  - libsystemd-shared
  - libsystemd0
  - libudev1
  - mdadm
  - netplan-generator
  - netplan.io
  - open-iscsi
  - open-vm-tools
  - python-apt-common
  - python3-apt
  - python3-distupgrade
  - python3-netplan
  - python3-tornado
  - rust-coreutils
  - snapd
  - sos
  - systemd-cryptsetup
  - systemd-resolved
  - systemd-sysv
  - systemd
  - thermald
  - tmux
  - ubuntu-kernel-accessories
  - ubuntu-minimal
  - ubuntu-pro-client-l10n
  - ubuntu-pro-client
  - ubuntu-release-upgrader-core
  - ubuntu-server-minimal
```

Figure 34. Rapport d'OPS01 : 48 mises à jour disponibles, dont `systemd`, `netplan.io` et `cloud-init` (réseau et démarrage).

```
TASK [Resume] ******************************************************************************
ok: [linux01] => {
    "msg": [
        "Mode : appliquer | mises a jour disponibles : 41",
        "Disques : / 47.1 %, /boot 11.2 %, /srv/data 5.3 %",
        "Redemarrage requis : non"
    ]
}

PLAY RECAP *********************************************************************************
linux01                    : ok=21   changed=2    unreachable=0    failed=0    skipped=1    resc
ued=0    ignored=0

adminops@ops01:~/infraready$ ansible linux01 -m command -a "resolvectl query win-dc01.infraready
.test" --ask-vault-pass
Vault password:
linux01 | CHANGED | rc=0 >>
win-dc01.infraready.test: 192.168.50.10                    -- link: ens33

-- Information acquired via protocol DNS in 2.6ms.
-- Data is authenticated: no; Data was acquired via local or encrypted transport: no
-- Data from: network
adminops@ops01:~/infraready$
```

Figure 35. LINUX01 en mode appliquer : 41 mises à jour installées, aucun redémarrage requis, le DNS du domaine résout toujours.

Machine	Avant	Intervention	Après
LINUX01	41 mises à jour, `/` à 47 %	41 paquets installés, services `running`	**0** mise à jour disponible
OPS01	48 mises à jour, `/` à 38 %	44 paquets installés ; incident `ansible-core` (annexe A1)	4 mises à jour différées par Ubuntu (phased updates)

PRÉCAUTION AVANT REDÉMARRAGE

Avant d'appliquer les mises à jour, on a vérifié que cloud-init ne gérait plus le réseau : sinon la mise à jour de `cloud-init` suivie d'un redémarrage aurait pu remettre le DNS sur la passerelle et rendre LINUX01 injoignable par son nom.

Windows : WIN-DC01

Le playbook `maintenance-windows.yml` pilote l'agent Windows Update natif (catégories sécurité, critiques et cumulatives). Il exige 10 Go libres sur C:, vérifie NTDS, DNS, Netlogon, ADWS, KDC, W32Time et WinRM avant et après, attend qu'Active Directory réponde après un redémarrage, et contacte le serveur par adresse IP pendant l'opération car son propre DNS est coupé au redémarrage.

```
TASK [Rechercher les mises a jour disponibles] *********************************************
ok: [win-dc01]

TASK [Installer les mises a jour (mode appliquer uniquement)] ******************************
skipping: [win-dc01]

TASK [Attendre que les services AD soient de nouveau prets] ********************************
ok: [win-dc01]

TASK [Relire l'etat du systeme] ************************************************************
ok: [win-dc01]

TASK [Retenir l'etat final] ****************************************************************
ok: [win-dc01]

TASK [Controler les services et le DNS apres intervention] *********************************
ok: [win-dc01] => {
    "changed": false,
    "msg": "Services critiques en marche et DNS du domaine operationnel"
}

TASK [Creer le dossier des preuves sur OPS01] **********************************************
ok: [win-dc01 -> localhost]

TASK [Ecrire le rapport de maintenance Windows] ********************************************
changed: [win-dc01 -> localhost]

TASK [Resume] ******************************************************************************
ok: [win-dc01] => {
    "msg": [
        "Mode : appliquer | mises a jour trouvees : 0",
        "Build : 10.0.26100.33650 -> 10.0.26100.33650",
        "Redemarrage en attente : non"
    ]
}

PLAY RECAP *********************************************************************************
win-dc01                   : ok=14   changed=1    unreachable=0    failed=0    skipped=1    resc
ued=0    ignored=0
```

Figure 36. Passage en mode appliquer : services AD/DNS en marche avant et après, build 26100.33650, aucun redémarrage en attente.

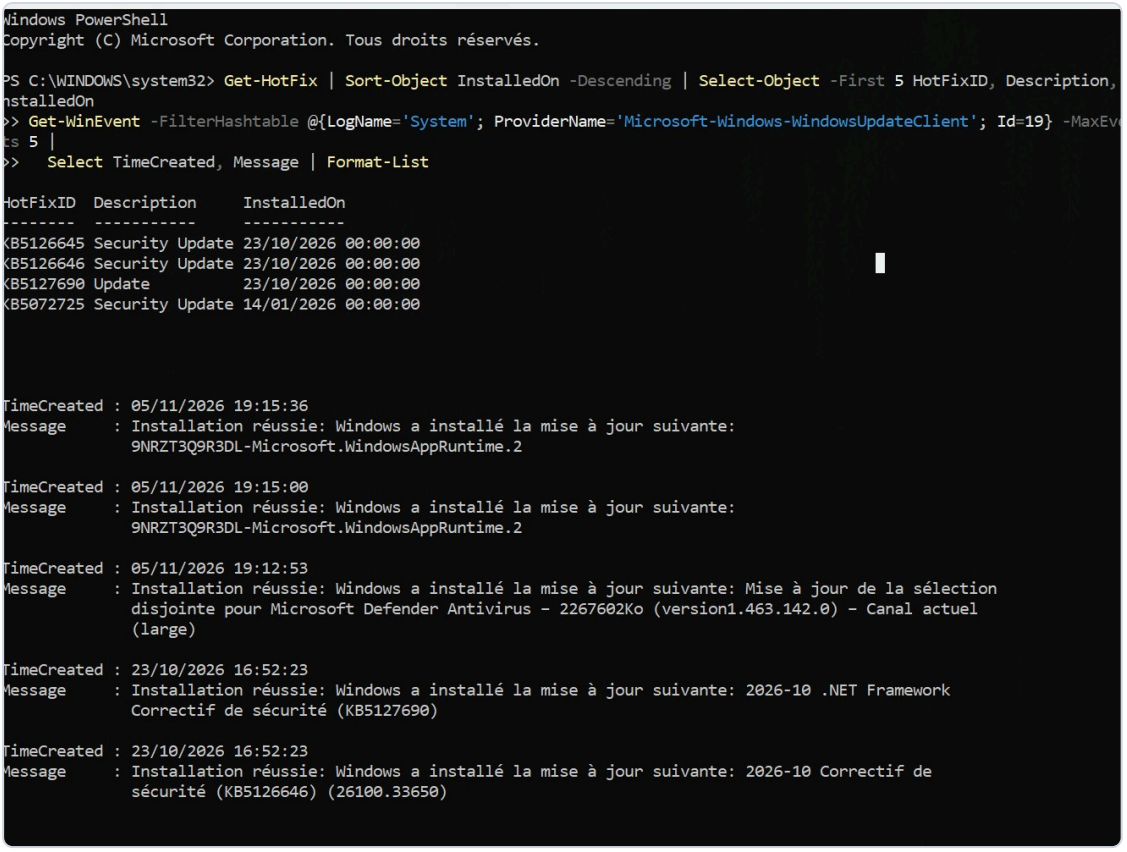


Figure 37. Historique : KB5126646 (cumulative de sécurité, build 26100.33650) et KB5127690 (.NET) installées le 23/10 à 16:52 ; mises à jour Defender et App Runtime installées automatiquement le 05/11.

CONSTAT

Windows installe aussi seul certaines mises à jour. Sur l'unique contrôleur de domaine, une installation automatique suivie d'un redémarrage couperait l'annuaire et le DNS du labo. Recommandation : stratégie Windows Update « télécharger et notifier », pour que l'installation reste déclenchée par la procédure.

8 Supervision avec Prometheus et Grafana

Collecte des métriques des trois serveurs, tableau de bord versionné, huit règles d'alerte, démonstration d'une alerte.

Composants

Machine	Composant	Port	Accès
LINUX01, OPS01	node_exporter (paquet Ubuntu), collecteur systemd activé pour nginx et ssh	9100	OPS01 seulement
WIN-DC01	windows_exporter 0.31.7 (MSI officiel)	9182	OPS01 seulement
OPS01	Prometheus, relevé toutes les 15 s	9090	Réseau du labo
OPS01	Grafana, source de données et tableau de bord provisionnés depuis le dépôt	3000	Réseau du labo

Tout est déployé par `playbooks/monitoring.yml`. La configuration (`monitoring/prometheus.yml`) et les règles sont validées par `promtool` au moment du déploiement. La dernière phase du playbook vérifie la chaîne de bout en bout : 4 cibles `up`, présence des 12 métriques utilisées par le tableau de bord, Grafana joignable et tableau de bord chargé.

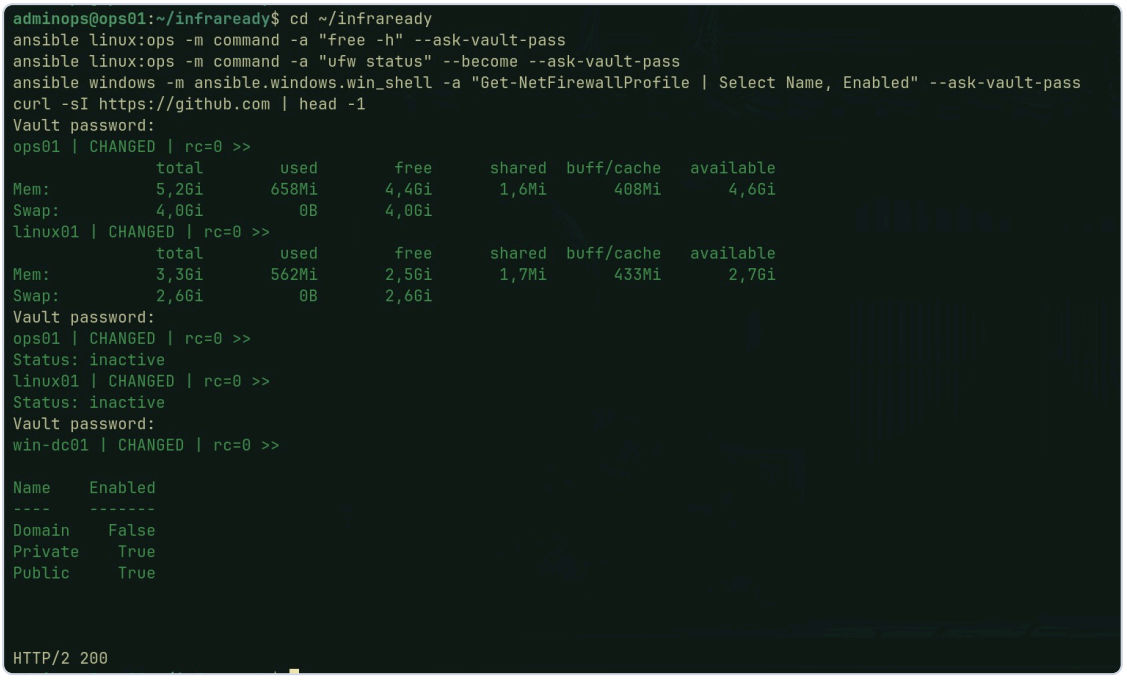


Figure 38. Prérequis vérifiés avant installation : mémoire disponible, état des pare-feux, accès à GitHub.

```
TASK [Echouer si une cible ne repond pas] ***************************************************************
ok: [ops01]

TASK [Interroger Prometheus pour chaque metrique attendue] **********************************************
ok: [ops01] => (item=node_cpu_seconds_total)
ok: [ops01] => (item=node_memory_MemAvailable_bytes)
ok: [ops01] => (item=node_filesystem_avail_bytes)
ok: [ops01] => (item=node_systemd_unit_state)
ok: [ops01] => (item=node_boot_time_seconds)
ok: [ops01] => (item=windows_cpu_time_total)
ok: [ops01] => (item=windows_memory_available_bytes)
ok: [ops01] => (item=windows_logical_disk_free_bytes)
ok: [ops01] => (item=windows_service_state)
ok: [ops01] => (item=windows_net_bytes_received_total)
ok: [ops01] => (item=windows_memory_physical_total_bytes)
ok: [ops01] => (item=windows_cs_physical_memory_bytes)
ok: [ops01] => (item=windows_system_boot_time_timestamp_seconds)
ok: [ops01] => (item=windows_system_boot_time_timestamp)

TASK [Lister les metriques presentes] *******************************************************************
ok: [ops01]

TASK [Controler que le tableau de bord aura toutes ses donnees] *****************************************
ok: [ops01] => {
    "changed": false,
    "msg": "Les 12 metriques du tableau de bord sont collectees"
}

TASK [Verifier que Grafana repond] **********************************************************************
ok: [ops01]

TASK [Verifier que la source Prometheus et le tableau de bord sont charges] *****************************
ok: [ops01]

TASK [Resume] *******************************************************************************************
ok: [ops01] => {
    "msg": [
        "Prometheus : http://192.168.50.20:9090  (cibles : /targets, alertes : /alerts)",
        "Grafana    : http://192.168.50.20:3000  (compte admin, tableau 'InfraReady - Vue d'ensemble')",
        "12 metriques collectees, 4 cibles up"
    ]
}

PLAY RECAP **********************************************************************************************
linux01                    : ok=5    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored
=0
ops01                      : ok=33   changed=15   unreachable=0    failed=0    skipped=0    rescued=0    ignored
=0
win-dc01                   : ok=6    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored
=0
```

Figure 39. Vérification de bout en bout : 12 métriques collectées, 4 cibles `up`, Grafana et tableau de bord opérationnels.

Tableau de bord

Le tableau de bord « InfraReady – Vue d'ensemble » contient huit panneaux : disponibilité des serveurs, alertes actives, utilisation CPU et mémoire, occupation des disques (seuil à 85 %), services critiques Linux et Windows, durée de fonctionnement et trafic réseau. Les métriques Linux et Windows sont affichées côte à côte.

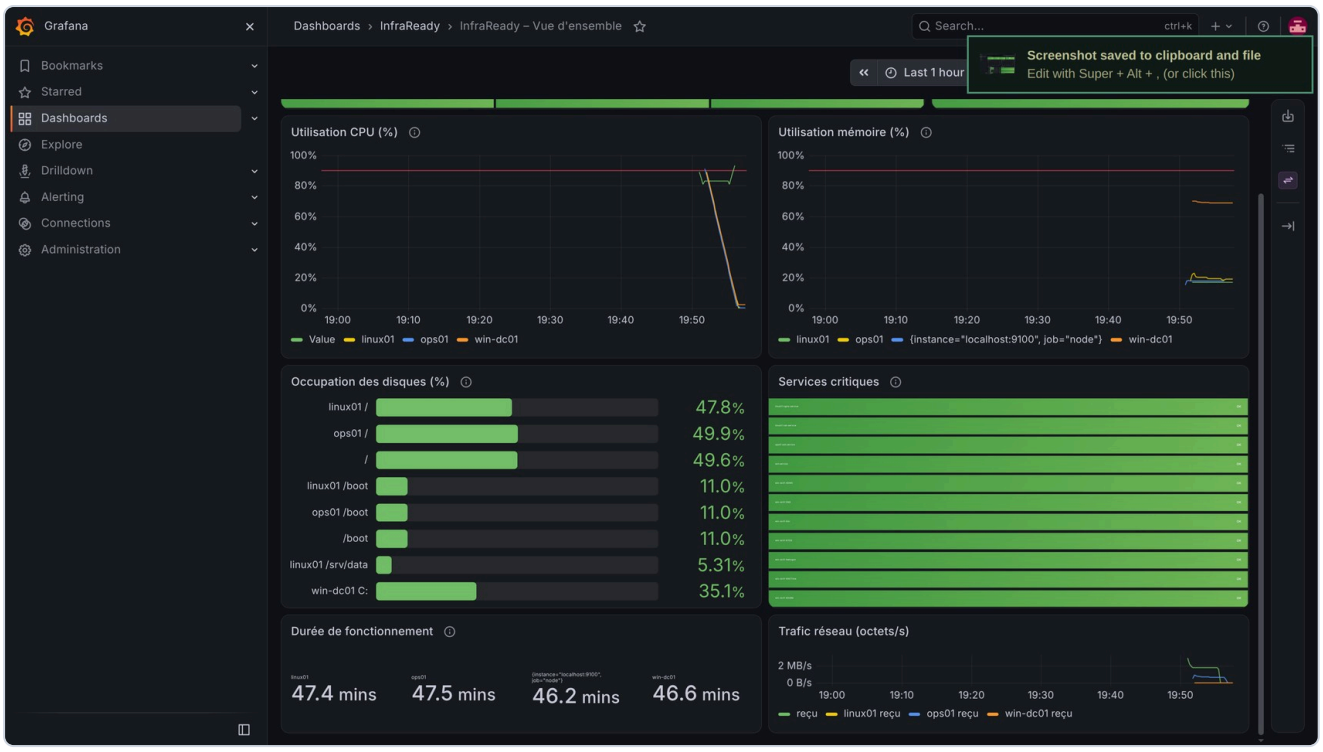


Figure 40. Vue complète du tableau de bord : métriques des trois serveurs.

Règles d'alerte

Règle	Condition	Délai	Sévérité
HoteInjoignable	`up == 0` (Linux ou Windows)	1 min	critique
DisqueLinuxPresquePlein	Système de fichiers > 85 %	2 min	avertissement
DisqueWindowsPresquePlein	C: > 85 %	2 min	avertissement
CPULinuxEleve / CPUWindowsEleve	CPU > 90 %	5 min	avertissement
MemoireLinuxElevee	Mémoire > 90 %	5 min	avertissement
ServiceLinuxArrete	`nginx.service` inactif	1 min	critique
ServiceWindowsArrete	NTDS, DNS, Netlogon ou KDC arrêté	1 min	critique

Les règles sont couvertes par des **tests unitaires** `promtool` (hôte éteint, disque à 90 %, nginx arrêté). Le délai `for:` évite qu'un simple redémarrage de service déclenche une fausse alerte.

Démonstration d'une alerte

nginx est arrêté volontairement sur LINUX01. En 15 secondes, le panneau *Services critiques* passe au rouge ; dans Prometheus, la règle `ServiceLinuxArrete` passe en *Pending*, puis en *Firing* au bout du délai d'une minute. Le service est ensuite relancé et l'alerte se résout d'elle-même.

```
adminops@ops01:~/infraready$ ansible linux01 -m ansible.builtin.service -a "name=nginx state=stopped" --become -
-ask-vault-pass
Vault password:
```

Figure 41. Arrêt volontaire de nginx depuis OPS01.

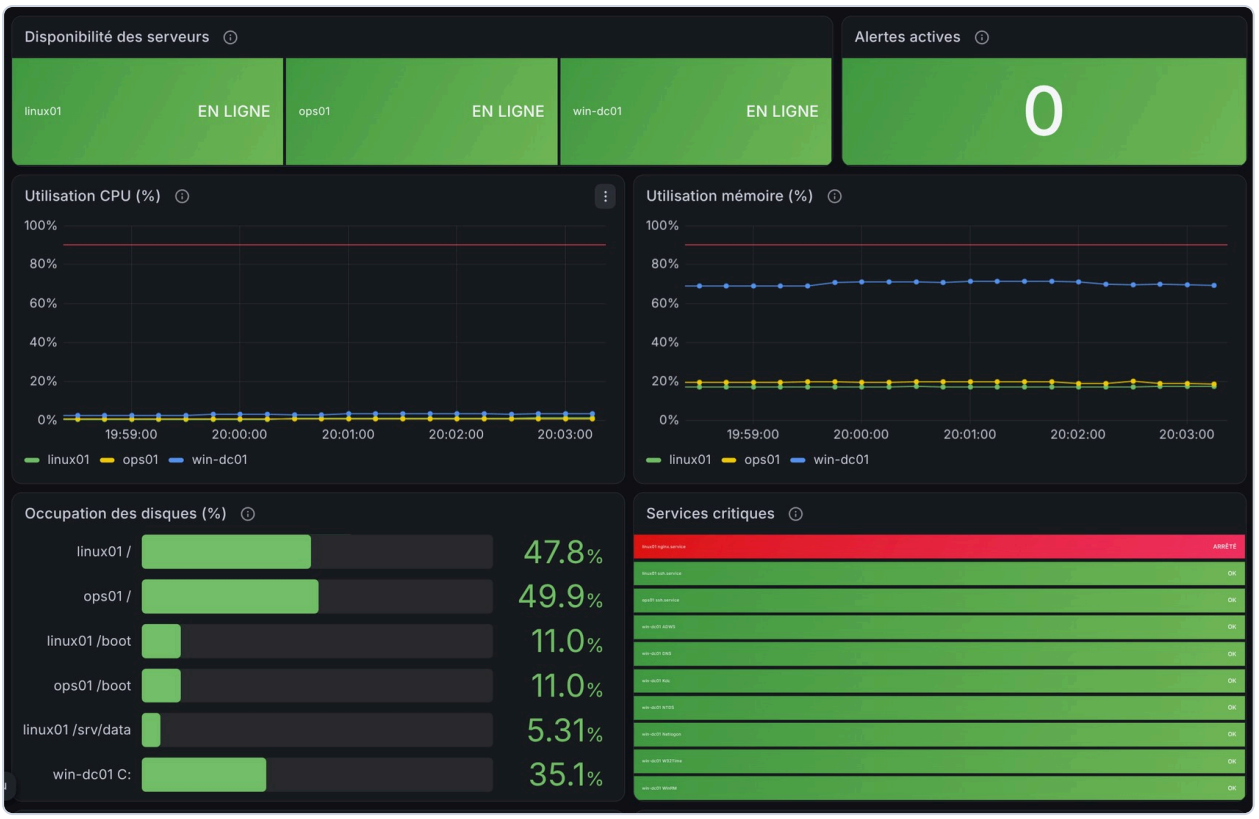


Figure 42. `linux01 nginx.service` détecté ARRÊTÉ ; les autres services Linux et Windows sont OK.

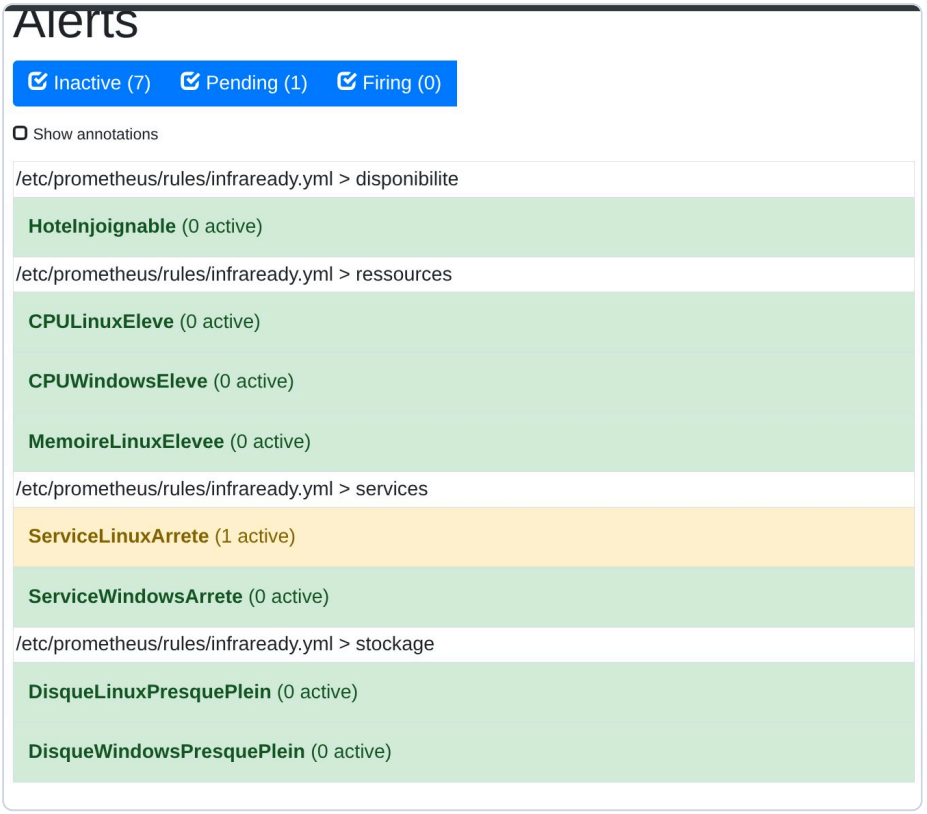


Figure 43. Prometheus : `ServiceLinuxArrete` en *Pending* (1 active) pendant le délai de confirmation.

9 GitHub et intégration continue

Dépôt organisé et versionné, workflow de validation à quatre jobs, contrôles rejoués en local en raison du blocage du compte GitHub.

Dépôt

Le dépôt suit la structure proposée par le sujet : `inventory/`, `playbooks/`, `monitoring/`, `scripts/`, `docs/`, `.github/workflows/`, complétée par `preuves/` (rapports générés) et `collections/requirements.yml`. Le travail des deux membres a été fusionné sans réécrire l'historique (pas de `push --force`).

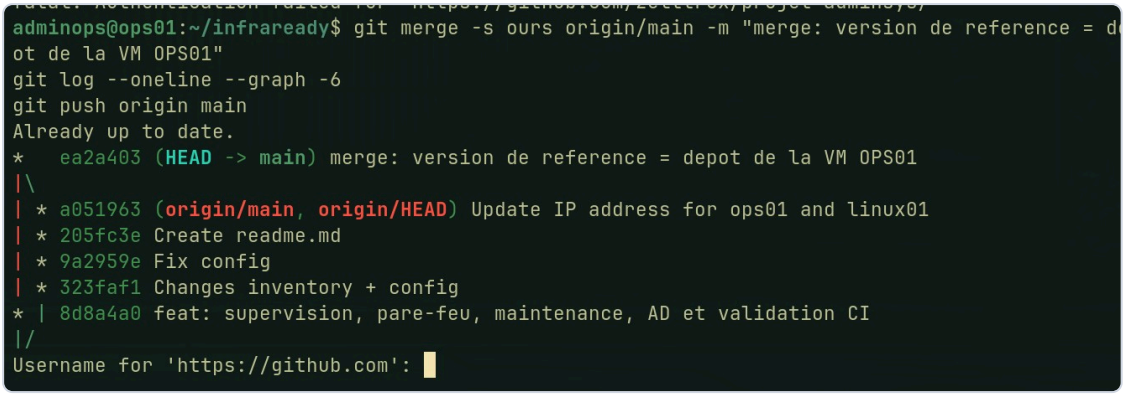


Figure 44. Historique : la version de référence (OPS01) est fusionnée avec les commits du binôme, l'historique est conservé.

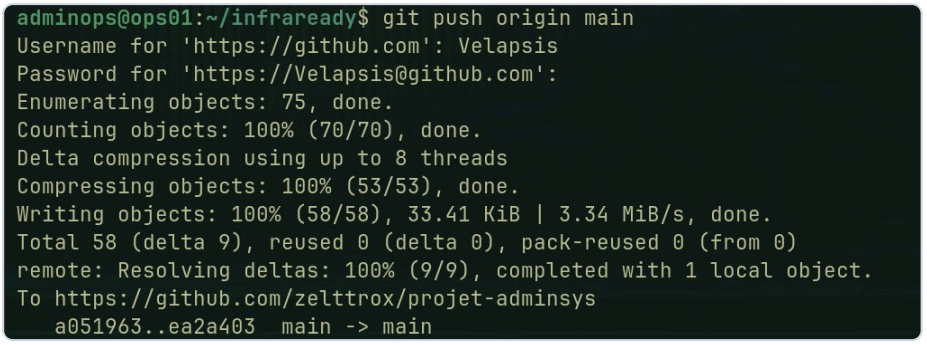


Figure 45. Envoi sur GitHub, authentifié par jeton d'accès personnel.

Workflow de validation

Job	Contrôles
Ansible	`yamllint`, `ansible-lint` (profil production), syntaxe de chaque playbook
Supervision	`promtool check config`, `check rules`, `test rules` ; validité du tableau de bord Grafana
Sécurité	Vault chiffrés, aucune clé privée, aucun mot de passe en clair (`verifier-secrets.sh`) ; `gitleaks` sur tout l'historique
PowerShell	Analyse syntaxique et PSScriptAnalyzer des scripts

Le workflow `.github/workflows/validation.yml` se déclenche à chaque push, à chaque pull request et à la demande. Il ne contacte aucune machine du labo : il valide le code, pas l'infrastructure.

Blocage de l'exécution sur GitHub

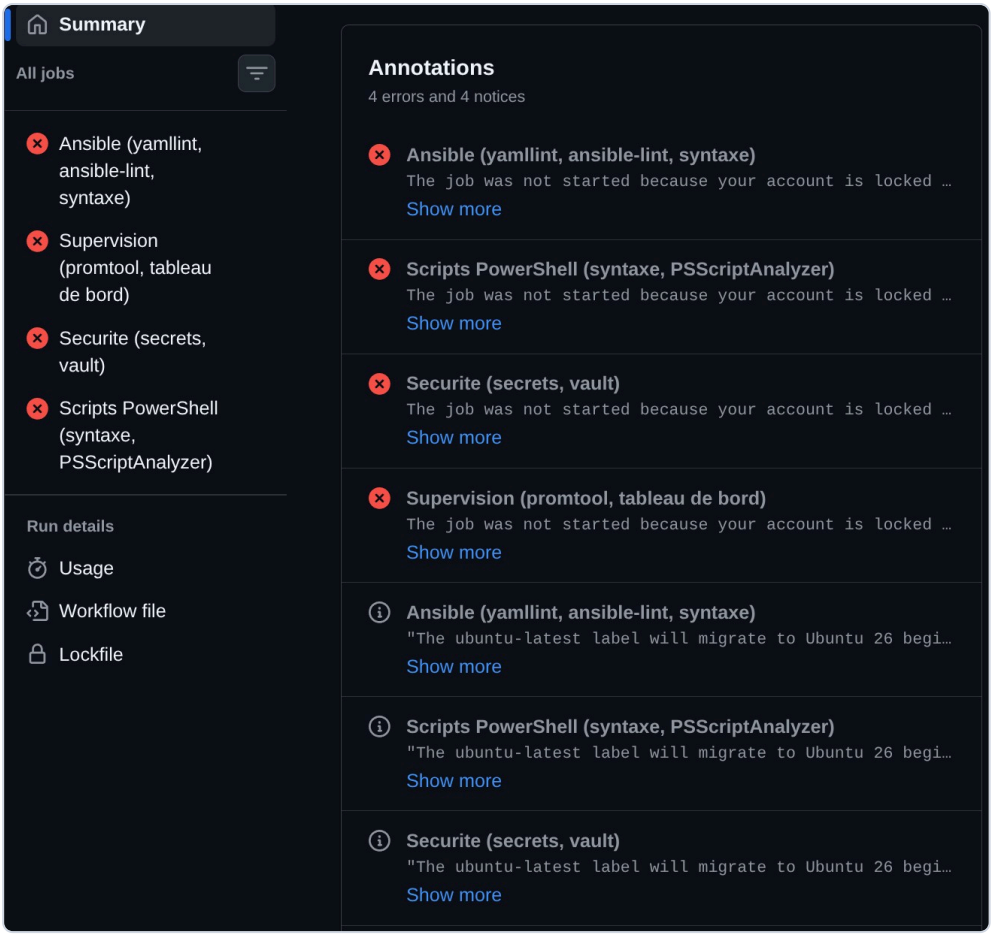


Figure 46. GitHub refuse de démarrer les quatre jobs : « The job was not started because your account is locked ».

> **FONCTIONNALITÉ NON TERMINÉE**
>
> Le compte propriétaire du dépôt ne bénéficie plus de l'offre éducation, et le binôme a choisi de ne pas y associer de moyen de paiement. Le workflow est écrit et validé (actionlint) ; il fonctionnera sans modification dès que GitHub Actions sera réactivé sur le compte (bouton *Re-run jobs*).

Contournement : validation locale

`scripts/ci/valider-local.sh` rejoue sur OPS01 exactement les contrôles du workflow et archive le résultat dans `preuves/`. Le premier passage a détecté deux défauts réels, corrigés ensuite : l'indentation des tâches de `nginx.yml` et l'absence de retour à la ligne final dans `update_servers.yml`.

```
adminops@ops01:~/infraready$ cat playbooks/linux/nginx.yml
---
- name: Installer et verifier Nginx sur LINUX01
  hosts: linux
  become: true
  tasks:
  - name: Installer le paquet nginx
    ansible.builtin.apt:
      name: nginx
      state: present
      update_cache: true
  - name: S'assurer que nginx est actif au demarrage
    ansible.builtin.service:
      name: nginx
      state: started
      enabled: true
```

Figure 47. Version initiale de `nginx.yml` : tâches non indentées sous `tasks:`, refusée par yamllint et ansible-lint.

preuves/G_validation_locale_2026-11-06_2013.txt — bilan avant correction

```
============================== BILAN ==============================
  Ansible : style YAML (yamllint)                                 ECHEC
  Ansible : bonnes pratiques (ansible-lint, profil production)    ECHEC
  Ansible : syntaxe de chaque playbook                            OK
  Supervision : configuration Prometheus                          OK
  Supervision : regles d'alerte                                   OK
  Supervision : tests unitaires des alertes                       OK
  Supervision : tableau de bord Grafana                           OK
  Securite : vault chiffres, cles privees, mots de passe en clair OK
  Securite : secrets dans tout l'historique Git (gitleaks)        OK
  PowerShell : syntaxe des scripts                                NON EXECUTE (outil pwsh absent : non insta
lle sur OPS01, controle effectue par la CI uniquement)
=====================================================================
RESULTAT : 2 controle(s) en ECHEC
Rapport : preuves/G_validation_locale_2026-11-06_2013.txt
```

preuves/G_validation_locale_2026-11-06_2015.txt — bilan après correction

```
============================== BILAN ==============================
  Ansible : style YAML (yamllint)                                 OK
  Ansible : bonnes pratiques (ansible-lint, profil production)    OK
  Ansible : syntaxe de chaque playbook                            OK
  Supervision : configuration Prometheus                          OK
  Supervision : regles d'alerte                                   OK
  Supervision : tests unitaires des alertes                       OK
  Supervision : tableau de bord Grafana                           OK
  Securite : vault chiffres, cles privees, mots de passe en clair OK
  Securite : secrets dans tout l'historique Git (gitleaks)        OK
  PowerShell : syntaxe des scripts                                NON EXECUTE (outil pwsh absent : non insta
lle sur OPS01, controle effectue par la CI uniquement)
=====================================================================
RESULTAT : tous les controles executes ont reussi
Rapport : preuves/G_validation_locale_2026-11-06_2015.txt
```

gitleaks a analysé les 9 commits de l'historique : **aucune fuite**. pwsh n'étant pas installé sur OPS01, l'analyse PowerShell est indiquée comme non exécutée en local ; elle reste assurée par le workflow.

10 Sécurité et limites connues

Moindre privilège, secrets chiffrés, pare-feu sur les trois machines, traçabilité ; et limites assumées, avec ce qui permettrait de les lever.

Mesures en place

Domaine	Mesures
Secrets	Ansible Vault (AES-256) ; mot de passe de vault fort et hors dépôt ; `no_log` sur les tâches sensibles ; contrôle automatique et gitleaks
Accès	SSH par clé ; WinRM HTTPS limité à OPS01 ; comptes Linux de service sans mot de passe ; comptes AD sans privilège
Moindre privilège	setgid sur le dossier web ; sudo du support limité à deux commandes ; exporters lisibles uniquement par OPS01
Pare-feu	ufw sur LINUX01 et OPS01 (entrée refusée par défaut, refus journalisés) ; profil Domaine activé sur WIN-DC01
Exploitation	Snapshots, garde-fous de formatage et de mise à jour, jamais de redémarrage du contrôleur par lui-même
Traçabilité	Journal sudo, journal du noyau (UFW BLOCK), événements 4720/4727/4728, rapports datés dans `preuves/`

Pare-feu

Le pare-feu est appliqué et vérifié par `playbooks/pare-feu.yml`. Le profil **Domaine** de WIN-DC01 était désactivé alors que le réseau est authentifié sur le domaine : le contrôleur fonctionnait donc sans pare-feu effectif. Le playbook teste aussi que les flux non prévus sont **refusés** (LINUX01 vers l'exporter Windows, WIN-DC01 vers l'exporter de LINUX01).

Règles ufw de LINUX01 et état du pare-feu Windows (premier passage)

```
        "Status: active",
        "Logging: on (low)",
        "Default: deny (incoming), allow (outgoing), disabled (routed)",
        "New profiles: skip",
        "",
        "To                         Action      From",
        "--                         ------      ----",
        "22/tcp                     ALLOW IN    192.168.50.0/24           # InfraReady - SSH",
        "80/tcp                     ALLOW IN    192.168.50.0/24           # InfraReady - site nginx",
        "9100/tcp                   ALLOW IN    192.168.50.20             # InfraReady - node_exporter"

TASK [Afficher l'etat] *********************************************************************
ok: [win-dc01] => {
    "msg": [
        "Profil reseau actif : DomainAuthenticated",
        "Avant : Domain=False, Private=True, Public=True"
    ]
```

```
PLAY RECAP *********************************************************************************
linux01                    : ok=10   changed=0    unreachable=0    failed=0    skipped=0    resc
ued=0    ignored=0
ops01                      : ok=10   changed=0    unreachable=0    failed=0    skipped=0    resc
ued=0    ignored=0
win-dc01                   : ok=5    changed=0    unreachable=0    failed=0    skipped=0    resc
ued=0    ignored=0
```

Figure 48. Second passage de `pare-feu.yml` : `changed=0` sur les trois machines, toutes les vérifications réussies.

Limites assumées

Limite	Risque	Pour la lever
Certificat WinRM auto-signé, validation désactivée	Usurpation de WIN-DC01 par un poste du même réseau	Certificat d'une AC interne (AD CS), validation activée
Authentification NTLM	Protocole plus ancien que Kerberos	Kerberos, avec OPS01 intégré au domaine
Ansible utilise un administrateur du domaine	Compromission d'OPS01 = compromission du domaine	Compte de service dédié, délégation limitée aux OU gérées
Clé SSH sans phrase de passe	Vol de la clé si OPS01 est compromis	Phrase de passe et `ssh-agent`
sudo-rs ne journalise pas les refus	Tentatives refusées invisibles	auditd, ou sudo classique
Grafana et Prometheus en HTTP	Mot de passe Grafana lisible sur le réseau du labo	Reverse proxy TLS
Mises à jour automatiques Windows	Redémarrage non planifié du contrôleur	Stratégie « télécharger et notifier »
GitHub Actions non exécuté	Pas de validation côté GitHub	Réactiver Actions sur le compte

11 Rapport d'incident

Absence de métriques de LINUX01 : incident simple, contrôlé et réversible, injecté volontairement puis diagnostiqué par couches.

Date	06/11/2026
Type	Absence de métriques d'un hôte (exemple du sujet)
Nature	Incident **injecté volontairement**, en solo ; diagnostic mené selon la démarche complète
Gravité	Moyenne : aucun service utilisateur interrompu, mais LINUX01 n'est plus supervisé

Injection : suppression de la règle ufw autorisant Prometheus sur le port 9100 de LINUX01, puis redémarrage de l'exporter. Le redémarrage est indispensable : ufw laisse passer les connexions **déjà établies**, et Prometheus réutilise une connexion persistante. Sans lui, la modification n'aurait eu aucun effet visible… jusqu'au prochain redémarrage, sans lien apparent avec la cause.

1. Symptôme observé



Figure 49. État normal à 20:21 : trois serveurs EN LIGNE.



Figure 50. Après l'injection : linux01 HORS LIGNE.

« Hors ligne » suggère une machine arrêtée, mais `up == 0` signifie seulement que Prometheus n'a pas pu lire l'exporter. L'alerte `HoteInjoignable` passe en *Pending* puis en *Firing* après une minute.

2. Tests et journaux consultés

#	Hypothèse testée	Résultat	Conclusion
a	Machine éteinte ou réseau coupé (`ping`)	0 % de perte	Machine et réseau OK
b	Services en panne (`curl` du site)	`HTTP/1.1 200 OK`	Utilisateurs non impactés
c	Administration perdue (`ansible -m ping`)	`pong`	SSH et Ansible OK
d	Exporter arrêté (`systemctl is-active`)	`active`	Le service tourne
e	Exporter défaillant (`curl localhost:9100`)	`200`	Répond **localement**

#	Hypothèse testée	Résultat	Conclusion
f	Flux bloqué (`nc` depuis OPS01)	`timed out`	**Filtrage entre les deux machines**
g	Règle de pare-feu (`ufw status numbered`)	Plus de règle `9100/tcp`	Cause trouvée
h	Journal du noyau (`UFW BLOCK`)	Paquets de 192.168.50.20 vers le port 9100 rejetés	Preuve
i	Origine (`journalctl -t sudo`)	Modules `ufw` exécutés par `adminops` via Ansible, horodatés	Modification attribuée et datée

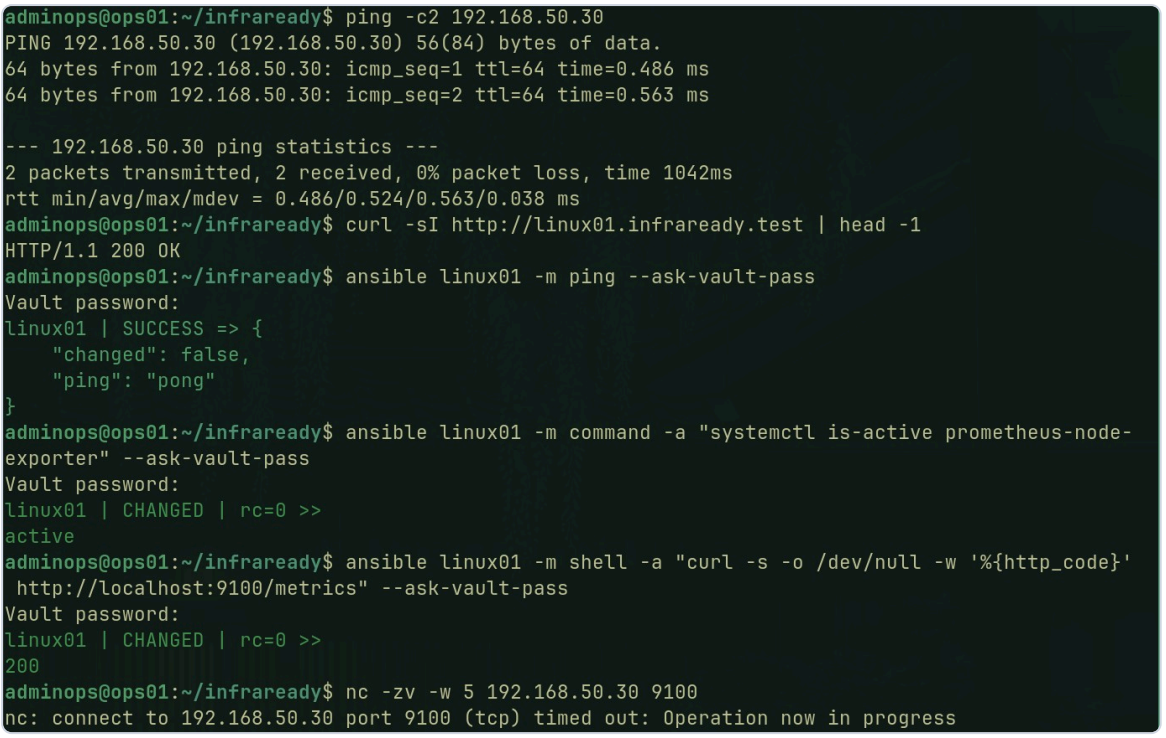


Figure 51. Tests a à f : tout fonctionne localement, mais la connexion depuis OPS01 vers le port 9100 expire.

```
adminops@ops01:~/infraready$ ansible linux01 -m command -a "ufw status numbered" --become --ask-
vault-pass
Vault password:
linux01 | CHANGED | rc=0 >>
Status: active

     To                         Action      From
     --                         ------      ----
[ 1] 22/tcp                     ALLOW IN    192.168.50.0/24           # InfraReady - SSH
[ 2] 80/tcp                     ALLOW IN    192.168.50.0/24           # InfraReady - site nginx
adminops@ops01:~/infraready$ ansible linux01 -m shell -a "journalctl -k --since '-15 min' | grep
 'UFW BLOCK' | grep 'DPT=9100' | tail -3" --become --ask-vault-pass
Vault password:
linux01 | CHANGED | rc=0 >>
nov. 06 20:24:51 linux01 kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:79:a9:a8:00:0c:29:42:50:
1e:08:00 SRC=192.168.50.20 DST=192.168.50.30 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=4104 DF PROTO=T
CP SPT=55448 DPT=9100 WINDOW=64240 RES=0x00 SYN URGP=0
nov. 06 20:25:11 linux01 kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:79:a9:a8:00:0c:29:42:50:
1e:08:00 SRC=192.168.50.20 DST=192.168.50.30 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=152 DF PROTO=TC
P SPT=33804 DPT=9100 WINDOW=64240 RES=0x00 SYN URGP=0
nov. 06 20:25:32 linux01 kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:79:a9:a8:00:0c:29:42:50:
1e:08:00 SRC=192.168.50.20 DST=192.168.50.30 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=5565 DF PROTO=T
CP SPT=46704 DPT=9100 WINDOW=64240 RES=0x00 SYN URGP=0
adminops@ops01:~/infraready$ ansible linux01 -m shell -a "journalctl _COMM=sudo --since '-30 min
' | grep -i ufw" --become --ask-vault-pass
Vault password:
[ERROR]: Task failed: Module failed: The command exited with a non-zero return code.
Origin: <adhoc 'shell' task>

{'action': 'shell', 'args': {'_raw_params': "journalctl _COMM=sudo --since '-30 min' | grep -i u
fw"}, 'timeout': [...]

linux01 | FAILED | rc=1 >>
The command exited with a non-zero return code.
```

Figure 52. Tests g et h : la règle 9100 a disparu ; le noyau journalise les tentatives rejetées toutes les ~20 s (rythme des relevés).

Le `timed out` plutôt qu'un `connection refused` s'explique par la politique ufw `deny`, qui jette les paquets sans répondre.

3. Cause identifiée

La règle ufw `9100/tcp ALLOW IN 192.168.50.20` avait été supprimée sur LINUX01 : la politique par défaut `deny (incoming)` bloquait Prometheus. La machine et ses services fonctionnaient normalement. Cause racine : modification manuelle du pare-feu hors du playbook qui en est la référence (dérive de configuration).

4. Correction appliquée

Pas de correction manuelle : l'état de référence est réappliqué par `ansible-playbook playbooks/pare-feu.yml`, qui ne modifie que ce qui a dérivé.

```
adminops@ops01:~/infraready$ ansible-playbook playbooks/pare-feu.yml --ask-vault-pass
nc -zv -w 5 192.168.50.30 9100
ansible-playbook playbooks/pare-feu.yml --ask-vault-pass
date
Vault password:

PLAY [Pare-feu Linux (ufw)] ****************************************************************

TASK [Gathering Facts] *********************************************************************
ok: [linux01]

TASK [Installer ufw] ***********************************************************************
ok: [linux01]

TASK [Autoriser les flux necessaires (avant activation)] ***********************************
ok: [linux01] => (item=22/tcp depuis 192.168.50.0/24 (SSH))
ok: [linux01] => (item=80/tcp depuis 192.168.50.0/24 (site nginx))
changed: [linux01] => (item=9100/tcp depuis 192.168.50.20 (node_exporter))

TASK [Politique par defaut - refuser en entree, autoriser en sortie] ***********************
ok: [linux01] => (item={'direction': 'incoming', 'politique': 'deny'})
ok: [linux01] => (item={'direction': 'outgoing', 'politique': 'allow'})
```

Figure 53. Correction : seule la règle `9100/tcp` est recréée (`changed`), les autres sont déjà conformes (`ok`).

5. Vérification du retour à la normale

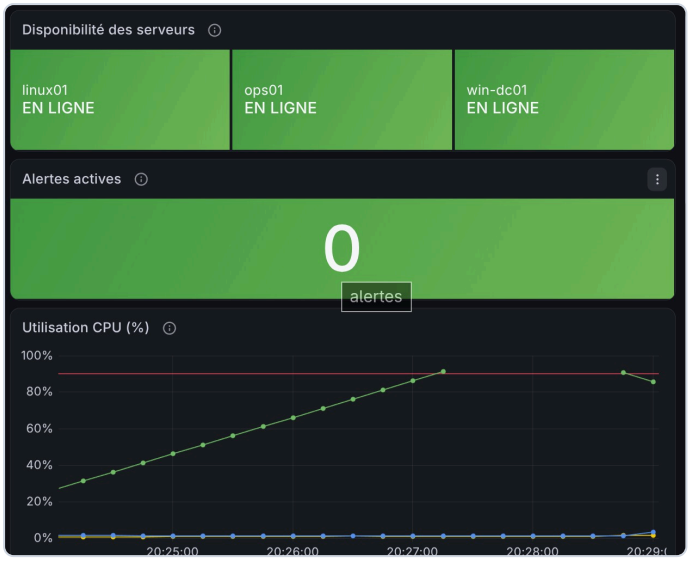


Figure 54. linux01 de nouveau EN LIGNE, 0 alerte active.

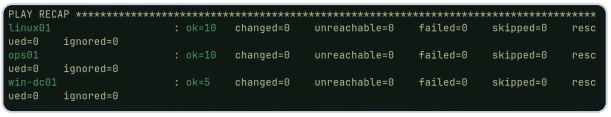


Figure 55. Second passage : `changed=0` partout.

```
adminops@ops01:~/infraready$ nc -zv -w 5 192.168.50.30 9100
Connection to 192.168.50.30 9100 port [tcp/*] succeeded!
```

Figure 56. La connexion d'OPS01 vers le port 9100 de LINUX01 aboutit de nouveau.

LECTURE DU GRAPHIQUE

Sur la courbe CPU, les valeurs de linux01 entre 20:24 et 20:29 correspondent à la période sans collecte : calculées sur une fenêtre de 5 minutes incomplète, elles ne sont pas exploitables et ne traduisent pas une charge réelle.

Chronologie et enseignements

Heure	Événement
~20:24	Injection : suppression de la règle et redémarrage de l'exporter

Heure	Événement
20:24:51	Premier paquet rejeté journalisé par ufw
~20:25	Cible DOWN dans Prometheus, linux01 HORS LIGNE dans Grafana
~20:26	Alerte `HoteInjoignable` déclenchée (délai d'une minute)
20:26 – 20:28	Diagnostic par couches (tests a à i)
~20:30	Correction par `pare-feu.yml`
20:31	Second passage `changed=0` : retour à la normale confirmé

- **Détection** en environ 1 minute (alerte automatique), **résolution** en environ 6 minutes.
- Le symptôme trompe : tester couche par couche évite de redémarrer une machine qui fonctionne.
- Tester localement puis à distance (e et f) isole immédiatement un problème réseau ou pare-feu.
- Une modification de pare-feu peut avoir un effet différé à cause des connexions établies.
- Le playbook est la référence : il corrige la dérive en une commande et prouve la conformité.

Annexe : incidents réels rencontrés pendant le projet

A1. Le contrôleur Ansible s'est supprimé lui-même (23/10, OPS01)

Symptôme	Le playbook de maintenance échoue (`Module result deserialization failed`), puis la commande `ansible` est introuvable
Tests / journaux	Sortie d'apt : `The following packages will be REMOVED: ansible-core …` ; `dpkg -l`
Cause	`ansible-core` était marqué comme dépendance automatique ; `autoremove: true` l'a supprimé pendant l'exécution sur le contrôleur lui-même
Correction	`sudo apt install ansible-core`, qui le marque comme installé manuellement
Vérification	`ansible --version`, collections intactes, trois hôtes joignables
Prévention	Paquets protégés, simulation `apt-get -s` et refus automatique — testé sur la sortie réelle de l'incident

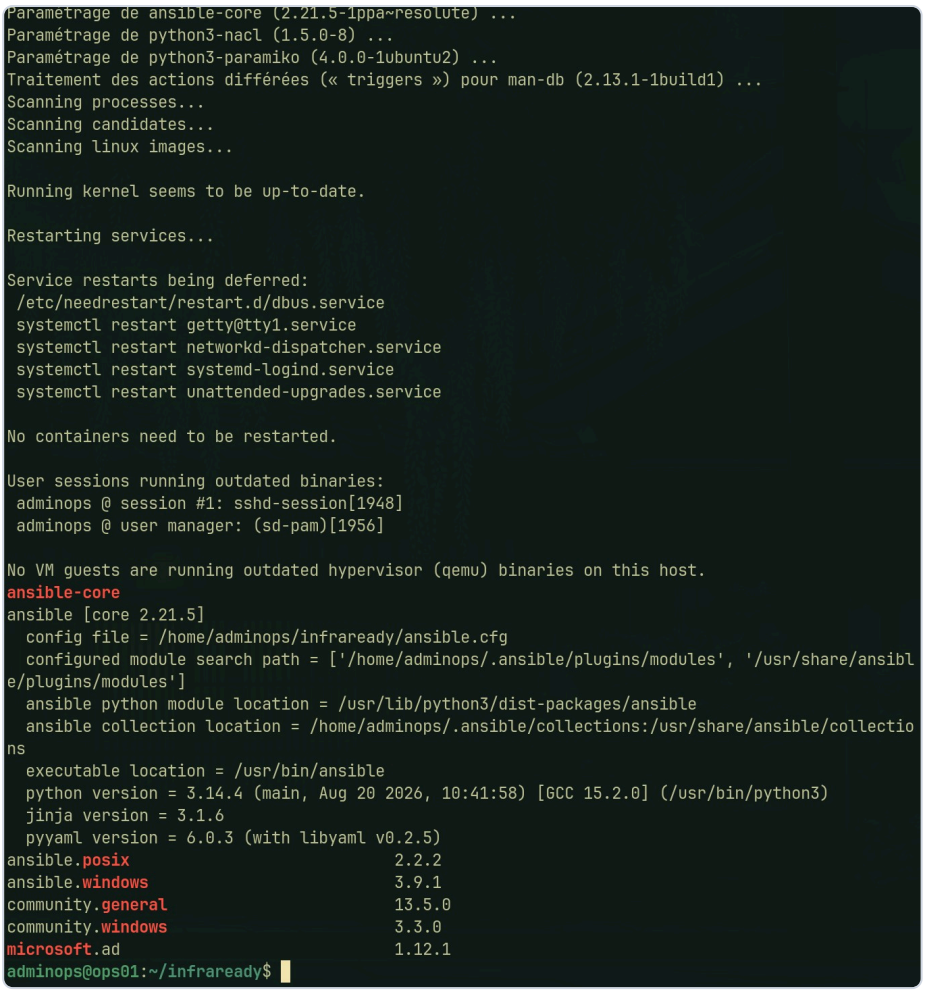


Figure 57. Réinstallation d'`ansible-core` : version 2.21.5, configuration du projet lue, collections présentes.

A2. Faux arrêt de nginx pendant un test de délégation sudo (23/10, LINUX01)

Symptôme	nginx arrêté juste après le test des droits de `support1`, censé ne pas pouvoir l'arrêter
Tests / journaux	`systemctl status` (arrêt propre, pas de plantage) ; `journalctl -u nginx` ; journal sudo
Cause	Le shell `support1` s'était refermé avant les commandes, exécutées en réalité par `adminlinux` : délégation non en cause
Correction	Relance de nginx ; test refait sans shell interactif (`sudo -u support1 sudo -n …`)
Vérification	`restart` autorisé, `stop` et `cat /etc/shadow` refusés (figure 23)



Figure 58. Aucun processus nginx, rien sur le port 80.

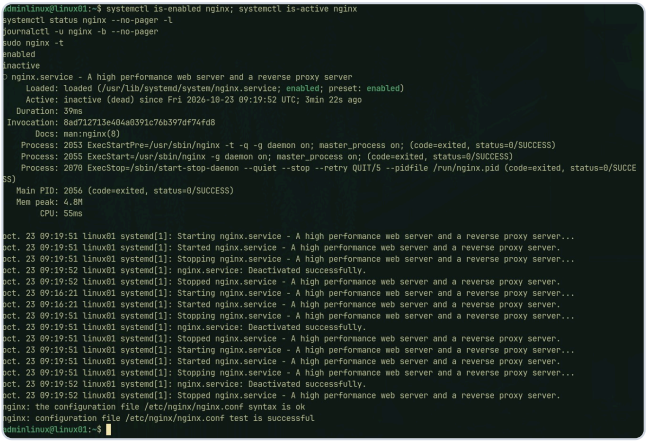


Figure 59. Arrêt propre demandé, configuration valide.

A3. Ansible bloqué à l'élévation de privilèges (22/10, LINUX01)

Symptôme	`Timeout (12s) waiting for privilege escalation prompt` sur tout playbook avec `become`
Tests	SSH par clé fonctionnel ; invite observée `[sudo: authenticate] Password:` ; version d'Ubuntu
Cause	Ubuntu 26.04 utilise sudo-rs, dont l'invite n'est pas reconnue par Ansible
Correction	`ansible_become_exe=sudo.ws` dans l'inventaire
Vérification	Playbook nginx : `ok=3 changed=1 failed=0` (figure 31)

12 Matrice critères → preuves

Chaque critère d'acceptation du sujet, relié à la preuve correspondante dans ce dossier ou dans le dépôt.

Partie	Critère	Preuve	Statut
A	Les trois machines démarrent correctement	Fig. 30, 39	Atteint
A	Les adresses IP sont documentées	Section 2, `docs/adressage-ip.md`, fig. 2–4	Atteint
A	Les tests de connectivité sont concluants	Fig. 6, 7	Atteint
A	Le DNS du domaine fonctionne	Fig. 15, 6	Atteint
B	Le domaine est opérationnel	Fig. 11, 15 (`dcdiag` sans erreur)	Atteint
B	Les comptes et groupes sont présents	Fig. 16, 17	Atteint
B	La résolution DNS est testée	Fig. 15 (directe, inverse, Internet)	Atteint
B	Les opérations réalisées sont documentées	Section 4, fig. 18	Atteint
C	Le service applicatif fonctionne	Fig. 20, 31	Atteint
C	Les permissions sont cohérentes avec le besoin	Fig. 23, 21, tests intégrés au playbook	Atteint
C	Le stockage est monté et documenté	Fig. 25, 26	Atteint
C	Les mises à jour et les journaux sont vérifiables	Fig. 35, rapports `preuves/E_maintenance_*`	Atteint
D	Les connexions Linux et Windows fonctionnent	Fig. 30, 29	Atteint
D	Les playbooks sont exécutables et rejouables	Fig. 22, 48 (`changed=0`)	Atteint
D	Aucun secret en clair	Fig. 28, section 9 (gitleaks : aucune fuite)	Atteint
E	Procédure de maintenance définie et appliquée	`docs/maintenance.md`, fig. 32, 35, 36	Atteint
E	Espace disque et services contrôlés	Fig. 33, 36	Atteint
E	Précautions avant redémarrage	Section 7, `docs/maintenance.md` §5	Atteint
F	Métriques Linux et Windows visibles dans Grafana	Fig. 40, 39	Atteint
F	Au moins un seuil ou une alerte démontré	Fig. 42, 43, 50	Atteint
G	Dépôt organisé, commits compréhensibles	Section 9, fig. 44	Atteint
G	Workflow GitHub Actions et preuve d'exécution	Fig. 46 (bloqué), validation locale section 9	Partiel
H	Incident diagnostiqué et corrigé (5 étapes)	Section 11, fig. 50–56	Atteint
§12	Les livrables sont complets et aucun secret n'est exposé	Ce dossier, `README.md`, `docs/`, gitleaks	Atteint